



Research paper

Imitation learning of a model predictive controller for real-time humanoid robot walking

Vítor G.B. de A. Porto ^a,*, Dicksiano C. Melo ^a, Marcos R.O.A. Maximo ^a,
Rubens J.M. Afonso ^b

^a Autonomous Computational Systems Lab (LAB-SCA), Computer Science Division, Aeronautics Institute of Technology, São José dos Campos, Brazil

^b Electronic Engineering Division, Aeronautics Institute of Technology, São José dos Campos, Brazil

ARTICLE INFO

Dataset link: <https://gitlab.com/itandroids/open-projects/imitation-learning-walking>

Keywords:

Humanoid robot walking
Imitation learning
Neural network
Model predictive control

ABSTRACT

Bipedal walking is an especially challenging task for humanoid robots, as it requires a robust controller to make a humanoid robot walk stably and react to disturbances. State-of-the-art algorithms have made use of model predictive controllers, but they require an exceedingly high computational cost and are often impractical to embed in a real robot's hardware. This paper contributes by showing how imitation learning may be employed to copy the behavior of a model predictive controller to a neural network, being able to predict the trajectory of the Center of Mass (CoM), as well as the position, rotation, and duration of each step. Our method was tested on a simplified simulation, a realistic full-body simulation, and on a real humanoid robot. The results showcase an algorithm that is close in terms of performance to the original controller, while requiring only a small fraction of its computational cost — with a speedup of 650 times, enabling it to be used in real time on real robots.

1. Introduction

Bipedal locomotion in Mobile Robotics remains a highly intricate and vital issue. The nonlinear and high-dimensional dynamics, natural instability, and complex physics present great challenges, especially in real-time control environments with restricted hardware resources. This has led to most successful models opting for simplified dynamics and reduced order controllers, as discussed in [Kajita et al. \(2003\)](#). While effective, such models face difficulties adapting to disturbances in walking patterns.

One of the key advantages of biped robots over robots with wheels is their ability to traverse uneven and unstructured terrain [Sygulla and Rixen \(2020\)](#). Additionally, when navigating diverse and uncontrolled spaces, a robot might bump, get stuck, or trip over something. For these reasons, a functional robot must be able to withstand and adapt to disturbances in real time.

To mitigate these challenges, researchers have turned to Control Theory methods to study the balance of humanoid agents ([Maximo et al., 2017](#)). Unfortunately, these techniques can be too stringent on limited real-time systems, particularly those that require online optimization like Model Predictive Controllers (MPC). Though MPC has been effective in robotic tasks ([Lopez et al., 2019](#); [Minniti et al., 2019](#)), its computational demands often make real-time application unfeasible.

Some adaptations, such as those in [Maximo et al. \(2020\)](#), allow MPC to run in real time but introduce constraints and potential performance losses.

Machine learning (ML) approaches, on the other hand, often concentrate the computational burden in the training phase, resulting in less demanding execution. These methods have the potential to bypass the limitations of prior model-based approaches by learning controllers directly from experience.

Reinforcement Learning (RL) is a common approach to employ ML for teaching robots to walk ([Melo et al., 2020, 2021, 2022](#)). However, RL may lead to extensive training time on real robots and even cause potential harm. Hence, this approach typically relies on simulators, though they may lead to unnatural behaviors and difficulty replicating results on actual robots due to the sim-to-real gap ([Salvato et al., 2021](#)). Following that, policies learned in simulations tend to be hard to reproduce in real robots due to the differences between the particularities of the simulator and the real world, so much so that many pieces of research are conducted in trying to narrow this gap ([Hietala et al., 2021](#); [Rosser et al., 2020](#)).

The contribution of this article is the behavior cloning of the gait generated by an MPC, using Deep Neural Networks (DNNs) to produce an end-to-end model, that receives the robot's inner states and

* Corresponding author.

E-mail addresses: vitor.giudice.porto@gmail.com (V.G.B. de A. Porto), dicksianomelo@gmail.com (D.C. Melo), mmaximo@ita.br (M.R.O.A. Maximo), rubensjm@ita.br (R.J.M. Afonso).

<https://doi.org/10.1016/j.engappai.2024.109919>

Received 16 May 2024; Received in revised form 3 December 2024; Accepted 19 December 2024

Available online 14 January 2025

0952-1976/© 2024 Published by Elsevier Ltd.

reference velocities at a given time and outputs the robot's actions required to perform the task in a safe and stable manner. The proposed method avoids the simulation gap as it learns directly from expert demonstrations. We expect the learned behavior to follow any reference in velocity stably while overcoming disturbances, inheriting the main strengths of the MPC, while doing it much faster, leveraging the speed of execution of the neural networks.

The expert used to teach the neural network-based controller is a model predictive controller – proposed in [Maximo and Afonso \(2020\)](#) – that automatically decides CoM trajectory and footstep positions, duration, and rotations while respecting actuation, geometry, and contact constraints. This controller leverages a Linear Inverted Pendulum Model (LIPM), a widely adopted framework in humanoid robotics ([Ding et al., 2022](#)), enhancing its applicability and generalizability across various humanoid robotic platforms.

The remainder of this article is organized as follows. Section 2 reviews the trending techniques and latest works in the field. Section 3 presents a background regarding humanoid robot walking and imitation learning. Section 4 shows our methodology for generating the expert's demonstrations and training the model. Section 5 discusses the results obtained with the proposed formulation, and Section 6 concludes and shares ideas for future work.

2. Literature review

Wieber et al. proposed a model predictive controller (MPC) that considers the Zero Moment Point (ZMP) constraints defined by the support polygon ([Wieber, 2006](#)). Then, Herdt et al. further enhanced this controller by adding the footstep positions as decision variables to allow automatic footstep placement ([Herdt et al., 2010a](#)).

Analytic methods based on heuristics were also proposed to adapt the step duration automatically. Missura and Behnke developed an omnidirectional gait control framework that uses heuristics to adapt both step position and duration when the robot is disturbed ([Missura and Behnke, 2013](#)). Furthermore, [Griffin et al. \(2017\)](#) presented an analytic algorithm for step placement and timing adaptation in a walking algorithm based on the concept of Divergent Component of Motion (DCM) ([Englsberger et al., 2015](#)).

Several studies have explored the inclusion of step duration as a decision variable within an optimization framework. For example, [Aftab et al. \(2012\)](#) have extended the footstep placement methodology proposed by [Herdt et al. \(2010b\)](#) with a nonlinear approach to encompass step duration. While the authors assert that real-time execution is attainable through a Sequential Quadratic Programming (SQP) solver, the evidence is limited to simplified simulation results without actual real-time performance evaluation. Similarly, [Kryczka et al. \(2015\)](#) developed an algorithm employing nonlinear optimization to adjust the footstep positions and durations in response to disturbances, as demonstrated in real robot experiments.

In another work, [Khadij et al. \(2016\)](#) formulated a two-stage method for the determination of step placement and timing. [Ibanez et al. \(2014\)](#) built upon the strategy introduced by [Herdt et al. \(2010a\)](#), employing Mixed-Integer Quadratic Programming (MIQP) to integrate footstep timing decisions.

Furthermore, [Bohórquez and Wieber \(2017\)](#) designed a controller incorporating step duration decisions. By applying principles from robust control, they aimed to ensure constraint satisfaction, favoring more conservative linear constraints rather than nonlinear ones.

Maximo et al., proposed an MIQP formulation to decide on CoM jerks, footstep positions, durations, and rotations while abiding by actuation, geometry, and contact constraints ([Maximo and Afonso, 2020](#)). To the best of our knowledge, this represents the pioneering work to concurrently include decisions on footstep timing, position, and rotation within an MIQP framework.

In light of the rise and subsequent popularity of deep learning methods, recent research has aimed to lessen the computational burden

of MPC by utilizing Neural Networks (NN), known for their relative efficiency in execution time ([Cavagnari et al., 1999](#); [Chen et al., 2018](#); [Karg et al., 2021](#); [Lovelett et al., 2018](#); [Åkesson and Toivonen, 2006](#); [Paulson and Mesbah, 2020](#); [Zhang et al., 2015](#); [Carius et al., 2020](#); [Reske et al., 2021](#); [Dong et al., 2022](#)). The common strategy is to employ NNs in mirroring expert demonstrations shaped by MPC ([Kumar et al., 2021](#)).

For instance, Cavagnari et al. demonstrate a method where a Non-linear Receding Horizon (NRH) control law is computed offline and then mirrored through an NN for online execution ([Cavagnari et al., 1999](#)). This design has been successfully applied to create an NRH controller for a seesaw device.

Chen et al. details a procedure to formulate an approximate explicit MPC law using NNs ([Chen et al., 2018](#)). Their method leverages a reinforcement learning policy gradient algorithm, incorporating system model insights to train the NN and assuring feasible control inputs through projections onto polytope regions.

Zhang et al. suggest merging MPC and RL in a guided policy search framework ([Zhang et al., 2015](#)), where MPC guides data generation in training. This information is utilized to train a DNN policy, and the result is evaluated on a simulated quadrotor, demonstrating the efficiency in obstacle avoidance behaviors. Meanwhile, Zamfirache et al. employed an NN-based RL with an actor-critic formulation to control a reference tracking device ([Zamfirache et al., 2023](#)).

Dong et al. present an MPC tracking scheme based on DNN ([Dong et al., 2022](#)), illustrating the sampling for training the network offline and enhancing the DNN-based MPC tracking with an Integral Module (IM). Their Hardware-in-the-Loop (HIL) simulation provides an alternative to traditional embedded MPC for complex systems.

Carius et al. propose an Imitation Learning (IL) method for controlling known model dynamical systems ([Carius et al., 2020](#)). Guided by MPC solutions and an innovative loss function, this work highlights improved constraint satisfaction in stabilizing various gaits on an actual walking quadrupedal robot in under 10 minutes.

Reske et al. describe an algorithm for a unified policy training that learns a walking robot's multiple gaits ([Reske et al., 2021](#)). Utilizing and extending MPC-Net ([Carius et al., 2020](#)), they introduce a new loss function to fine-tune single- and multi-gait policies, achieving improvements with a Mixture-of-Experts Network (MEN).

The present article seeks to augment the existing literature by exploring a lesser-studied area within humanoid robot walking. It capitalizes on imitation learning to replicate the decision-making process of a robust MPC's walking pattern generator. This method enables leveraging specific aspects of the robot such as its symmetry, simplifying the learning for the agent. Additionally, the focus on a higher-level pattern generator allows for the integration of other low-level control loops, such as gravity compensation, improving the overall walking algorithm.

3. Background

3.1. Humanoid robot walking

Walking is a cyclical movement in which feet move alternately, ensuring that at least one foot remains in contact with the ground at all times ([Kajita et al., 2014](#); [Wieber et al., 2016](#)). In the context of legged locomotion, this action entails a multidomain hybrid dynamical system in which sudden changes in continuous dynamics are prompted by discrete occurrences. A full walking cycle is comprised of two distinct single support stages, along with two double support stages. Consequently, a step symbolizes half of this cycle, encompassing one single support phase coupled with one double support phase.

When examining the robot's dynamics, the characteristics between single and double support phases differ significantly. During the double support phase, both feet maintain ground contact, rendering them immobile without slipping, hence constituting what is named a closed kinematic chain. Conversely, the single support phase permits the swing

foot to be lifted from the ground, allowing free movement, which corresponds to an open kinematic chain.

Furthermore, the two phases are notably distinguishable in terms of the support polygon, a vital factor in stability. The support polygon is delineated as the convex hull that encompasses all ground contact points. In a double support phase, the support polygon includes a broad area shaped by the convex hull of both feet's vertices. In contrast, the single support phase's support polygon is confined to the supporting foot (or its portion in ground contact). As a result, the double support phase entails a larger margin for stability to the robot compared to single support.

The most widely used biped stability criterion is the ZMP (Vukobratović and Borovac, 2004). The ZMP coincides with the point on the ground where the ground reaction forces must act to balance the biped. ZMP may be regarded as the dynamic equivalent of the CoM projection: if the ZMP is within the support polygon in a given time, dynamic balance is ensured. Consequently, keeping the ZMP within the support polygon at all times suffices for ensuring stability and allows the robot to overcome the speed limit of static walking. Nevertheless, ZMP stability is over-restrictive: if the ZMP leaves the support polygon in a given instant, the robot does not fall instantly, and further movements may recover balance.

Dealing with the full-body dynamics of a fully-actuated humanoid robot is unfeasible due to its large dimensionality and high nonlinearity. Therefore, most successful walking controllers deal with reduced-order dynamic models. The most popular of these models is the Linear Inverted Pendulum Model (LIPM) (Kajita et al., 2001), which is based on the following assumptions: (i) the robot is walking in a horizontal plane, (ii) the CoM is kept at a constant height h , (iii) the angular momentum rate may be neglected, and (iv) the ZMP is within the support polygon at all times. By employing the Newton–Euler equations of rigid body dynamics and these assumptions, we may derive the LIPM's equations in x and y :

$$\mathbf{r}_{CoM} - \frac{h}{g} \ddot{\mathbf{r}}_{CoM} = \mathbf{r}_{ZMP}, \quad (1)$$

where $\mathbf{r}_{ZMP} = [x_{ZMP}, y_{ZMP}]^T$ and $\mathbf{r}_{CoM} = [x_{CoM}, y_{CoM}]^T$ are the ZMP and CoM positions on the horizontal plane, respectively, and g is the acceleration of gravity. For a complete derivation of this equation, we refer the interested reader to Wieber et al. (2016). Control methods that reason about reduced-order models typically decide CoM trajectory. Then, an additional algorithm computes actual joint commands through inverse kinematics or inverse dynamics to attain the desired CoM trajectory (Kajita et al., 2014).

3.2. Behavior cloning

Imitation Learning is a technique where the agent learns a policy from demonstrations (Duan et al., 2017). It can basically be divided into Inverse Reinforcement Learning (IRL) and Behavior Cloning (Torabi et al., 2018). IRL learns a reward function from expert demonstrations (Ng and Russell, 2000). Then, it can be used to recover a policy by running the standard reinforcement learning framework. Behavior Cloning (Bain and Sammut, 1995) applies imitation learning as a supervised learning problem, where the policy is learned by state-action tuples provided by the expert and used to train a classifier or regressor.

Comparatively speaking, in both methods, there is no interactive demonstrator and both use pre-sampled demonstrations. Also, only IRL has direct access to the learning environment, as summarized in Table 1.

In summary, IRL can be divided into two separate parts: obtaining the reward function and using the optimized reward function to optimize the policy. This makes it harder to apply to some scenarios in which the direct interaction with the environment is costly, as in each iteration the model must get the next state from it. So, although

Table 1

Comparison between Behavior Cloning and IRL.

	Behavior Cloning	IRL
Direct Policy Learning	Yes	No
Reward Learning	No	Yes
Access to Environment	No	Yes
Interactive Demonstrator	No	No
Pre-collected Demonstrations	Yes	Yes

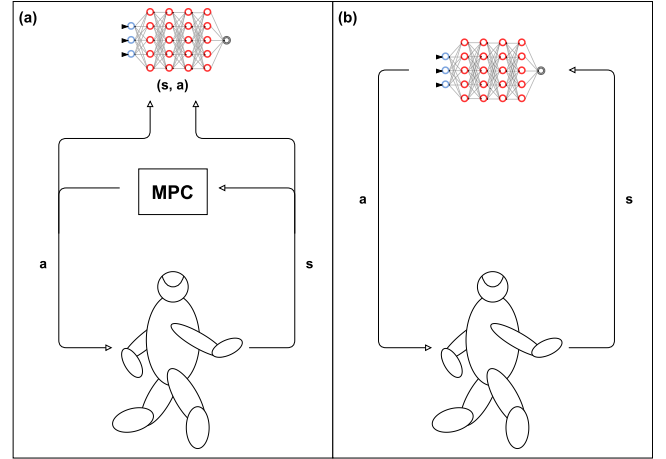


Fig. 1. Proposed methodology. (a): Training phase – MPC represents the Model Predictive Controller, s represents the robot's state, a represents the MPC's output, (s, a) represents the state-action pair sampled to train the NN through supervised learning. (b): Deployment phase – s represents the robot's state, a represents the NN's output.

the method uses a pre-collected demonstration set, as does behavior cloning, the need to directly interact with the environment in order to optimize the policy makes IRL comparatively more costly than behavior cloning, which can collect all the needed data beforehand. For that reason, we opted to use behavior cloning for the IL controller.

The architecture proposed in this work can then be seen in Fig. 1. In the dataset generation phase – Fig. 1(a), a simulated robot walks using the MPC described in Section 4.1. During simulation, the state vector and the MPC output were recorded as tuples (s, a) in order to train an NN as described in Section 4.3. In the deployment phase – Fig. 1(b), the trained NN replaces the MPC. The performances of both approaches were compared as described in Section 5.3.

4. Methodology

This section provides a walkthrough of the methods that formed the foundation of our proposed method. It describes the use of the Mixed Integer Quadratic Programming (MIQP) formulation on the Model Predictive Controller (MPC) that serves as the expert for the Imitation Learning (IL) controller. Moreover, it describes how to make the robot walk using the IL controller and how to generate the dataset used during training. Finally, a description of the resulting model and the methods employed to tune the hyperparameters is provided.

The codebase used to develop this work can be found on <https://gitlab.com/itandroids/open-projects/imitation-learning-walking>

4.1. Humanoid walking with MIQP

In this section, we present the model-based walking controller used as a source of data for the imitation task. This controller is based on previous works (Maximo et al., 2016; Maximo, 2017; Maximo and Afonso, 2020). It is highly flexible, as given a reference CoM velocity $\mathbf{v}_r = [\dot{x}_r, \dot{y}_r, \dot{\theta}_r]^T$ (with $\dot{x}_r$, $\dot{y}_r$, and $\dot{\theta}_r$ being the forward, sideways, and rotational reference velocities), it automatically decides CoM jerks,

footstep positions, durations, and rotations while respecting actuation, geometry, and contact constraints. Moreover, simulation results show that it can withstand strong disturbances (Maximo and Afonso, 2020). This algorithm is attained by framing the walking problem as a constrained optimization problem through model predictive control (MPC), specifically using mixed-integer quadratic programming (MIQP):

$$\mathbf{u}^*[k] = \arg \min_{\mathbf{u}[k] \in \mathbb{R}^{N_{con}} \times \{0,1\}^{N_{bin}}} \frac{1}{2} \mathbf{u}^\top [k] \mathbf{H}[k] \mathbf{u}[k] + \mathbf{f}^\top [k] \mathbf{u}[k] \quad (2)$$

subjected to $\mathbf{A}_{ineq}[k] \mathbf{u}[k] \leq \mathbf{b}_{ineq}[k]$

where a new optimization problem must be solved at every time sample k . The decision variables vector $\mathbf{u}[k]$ comprises N_{con} continuous variables and N_{bin} binary variables. The quadratic cost function is defined by $\mathbf{H}[k]$, a symmetric positive-definite matrix, and $\mathbf{f}[k]$, a real-valued vector. Moreover, the constraints are encoded by matrix $\mathbf{A}_{ineq}[k]$ and vector $\mathbf{b}_{ineq}[k]$.

The downside of the approach is its high computational burden, which is incompatible with real-time execution on an embedded computer, even when using state-of-the-art MIQP solvers, such as Gurobi (Gurobi Optimization, Inc., 2015). Indeed, in Maximo and Afonso (2020), a mean computation time for solving the optimization problem of around 330 ms was obtained, higher than the sample time of 100 ms considered for the walking controller. The algorithm was executed on a 2.6 GHz i7-9750H hexa-core notebook. Therefore, we propose approximating this model-based controller using a neural network through behavior cloning.

Since many implementation details of the optimization mechanisms are irrelevant to the imitation task, we opted for a concise description of the MIQP controller here. We refer the interested reader to Maximo and Afonso (2020) for a more detailed explanation.

4.1.1. Predicting the future

In MPC, a model of the dynamical system is integrated over a fixed horizon to predict how a sequence of inputs will affect future states. Herein, we present how this can be done for walking dynamics. We will derive the equations only for the x coordinate since it can be analogously extended to the y coordinate.

First, we consider that the robot controls the jerk (i.e., the derivative of acceleration) $\ddot{x}$ of its CoM, a standard assumption in LIPM (Naveau et al., 2017; Bohórquez and Wieber, 2017, 2018). Therefore, the CoM evolution follows

$$\frac{d}{dt} \begin{bmatrix} x \\ \dot{x} \\ \ddot{x} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \ddot{x} \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \ddot{x}. \quad (3)$$

We discretize this dynamic system by assuming zero-order hold (ZOH) within a time step of duration T :

$$\mathbf{x}[k+1] = \begin{bmatrix} 1 & T & T^2/2 \\ 0 & 1 & T \\ 0 & 0 & 1 \end{bmatrix} \mathbf{x}[k] + \begin{bmatrix} T^3/6 \\ T^2/2 \\ T \end{bmatrix} \ddot{x}[k], \quad \mathbf{x}[k] = \begin{bmatrix} x[k] \\ \dot{x}[k] \\ \ddot{x}[k] \end{bmatrix}. \quad (4)$$

Using (1), one may write:

$$\mathbf{r}_{ZMP,x}[k] = [1 \quad 0 \quad -h/g] \mathbf{x}[k]. \quad (5)$$

Thus, we may predict future system behavior over a horizon of N time steps as

$$\mathbf{X}[k] = \begin{bmatrix} \mathbf{x}[k+1|k] \\ \vdots \\ \mathbf{x}[k+N|k] \end{bmatrix} = \mathbf{P}_{ps} \mathbf{x}[k] + \mathbf{P}_{pu} \ddot{\mathbf{X}}[k], \quad (6)$$

$$\dot{\mathbf{X}}[k] = \begin{bmatrix} \dot{\mathbf{x}}[k+1|k] \\ \vdots \\ \dot{\mathbf{x}}[k+N|k] \end{bmatrix} = \mathbf{P}_{vs} \mathbf{x}[k] + \mathbf{P}_{vu} \ddot{\mathbf{X}}[k], \quad (7)$$

$$\ddot{\mathbf{X}}[k] = \begin{bmatrix} \ddot{\mathbf{x}}[k+1|k] \\ \vdots \\ \ddot{\mathbf{x}}[k+N|k] \end{bmatrix} = \mathbf{P}_{as} \mathbf{x}[k] + \mathbf{P}_{au} \ddot{\mathbf{X}}[k], \quad (8)$$

$$\mathbf{Z}_x[k] = \begin{bmatrix} \mathbf{r}_{ZMP,x}[k+1|k] \\ \vdots \\ \mathbf{r}_{ZMP,x}[k+N|k] \end{bmatrix} = \mathbf{P}_{zs} \mathbf{x}[k] + \mathbf{P}_{zu} \ddot{\mathbf{X}}[k], \quad (9)$$

where:

$$\ddot{\mathbf{X}}[k] = \begin{bmatrix} \ddot{x}[k|k] \\ \vdots \\ \ddot{x}[k+N-1|k] \end{bmatrix}. \quad (10)$$

We refer the interested reader to Maximo and Afonso (2020) for expressions for the matrices $\mathbf{P}_{ps}$, $\mathbf{P}_{pu}$, $\mathbf{P}_{vs}$, $\mathbf{P}_{vu}$, $\mathbf{P}_{as}$, $\mathbf{P}_{au}$, $\mathbf{P}_{zs}$, and $\mathbf{P}_{zu}$.

4.1.2. Decision variables

Denote N_f and N_r as the maximal number of footsteps within the prediction horizon and the number of discrete values each foot rotation can assume, respectively. The MIQP formulation has the following decision variables:

$$\mathbf{u}[k] = \begin{bmatrix} \ddot{\mathbf{X}}[k] \\ \ddot{\mathbf{Y}}[k] \\ \mathbf{X}_f^b[k] \\ \mathbf{Y}_f^b[k] \\ \boldsymbol{\theta}_f^b[k] \\ \mathbf{B}_u[k] \\ \mathbf{X}_f^{aux}[k] \\ \mathbf{Y}_f^{aux}[k] \\ \boldsymbol{\theta}_f^{aux}[k] \\ \boldsymbol{\theta}_f^r[k] \\ \mathbf{S}_r[k] \\ \mathbf{B}_s[k] \end{bmatrix} \in \mathbb{R}^{4(N+N_f)} \times \{0,1\}^{N_f(2N-N_f+1)/2+N_f+N_r(N+N_f)}, \quad (11)$$

where:

- $\ddot{\mathbf{X}}[k] \in \mathbb{R}^N$ is the future CoM jerk in the x axis within the prediction horizon.
- $\ddot{\mathbf{Y}}[k] \in \mathbb{R}^N$ is the future CoM jerk in the y axis within the prediction horizon.
- $\mathbf{X}_f^b[k] \in \mathbb{R}^{N_f}$ contains the support foot positions in the x axis at each footstep.
- $\mathbf{Y}_f^b[k] \in \mathbb{R}^{N_f}$ contains the support foot positions in the y axis at each footstep.
- $\boldsymbol{\theta}_f^b[k] \in \{0,1\}^{N_f N_r}$ determines the active support foot rotation at each footstep.
- $\mathbf{B}_u[k] \in \{0,1\}^{N_f(2N-N_f+1)/2}$ determines which footstep is active at each time step.
- $\mathbf{X}_f^{aux}[k] \in \mathbb{R}^N$ contains the support foot positions in the x axis at each time step.
- $\mathbf{Y}_f^{aux}[k] \in \mathbb{R}^N$ contains the support foot positions in the y axis at each time step.
- $\boldsymbol{\theta}_f^{aux}[k] \in \{0,1\}^{N_f N_r}$ chooses the active support foot rotation at each time step.
- $\boldsymbol{\theta}_f^r[k] \in \mathbb{R}^{N_f}$ collects reference footstep rotations.
- $\mathbf{S}_r[k] \in \mathbb{R}^{N_f}$ collects reference footstep durations.
- $\mathbf{B}_s[k] \in \{0,1\}^{N_f}$ holds binary variables which indicate if a footstep had its reference step duration modified to take into account that it is at the end of the prediction horizon.

Notice that $\ddot{\mathbf{X}}[k]$, $\ddot{\mathbf{Y}}[k]$, $\mathbf{X}_f^b[k]$, $\mathbf{Y}_f^b[k]$, $\boldsymbol{\theta}_f^b[k]$, and $\mathbf{B}_u[k]$ represent decisions executed by the robot, so they will be related to outputs of the neural network. On the other hand, $\mathbf{X}_f^{aux}[k]$, $\mathbf{Y}_f^{aux}[k]$, $\boldsymbol{\theta}_f^{aux}[k]$, $\boldsymbol{\theta}_f^r[k]$, $\mathbf{S}_r[k]$, and $\mathbf{B}_s[k]$ are auxiliary variables used to encode the optimization problem as an MIQP. Finally, a typical MPC scheme plans control inputs for the whole prediction horizon; however, only the first action is executed. In the next time step, the controller recomputes the values for

the remaining actions due to updated feedback. In practice, the neural network may refrain from learning the whole sequence of commands but only the commands to be executed at the current time step, in view of the reiteration at every sample time, characteristic of the operation in closed loop.

To allow the variable duration of the footsteps, $\mathbf{B}_u[k]$ is remapped to a matrix of binary variables $\mathbf{U}_b[k] \in \{0, 1\}^{N \times N_f}$ whose elements are

$$(\mathbf{U}_b[k])_{i,j} = \begin{cases} 1, & \text{if the } j\text{th footstep is active at the } i\text{th time step} \\ 0, & \text{otherwise} \end{cases}, \quad (12)$$

where a footstep is active if it is the support foot position at the given time step. These binary variables encode a mapping between time steps (discrete instants of time) and active footsteps within the prediction horizon, where time steps and footsteps change in the rows and columns of matrix $\mathbf{U}_b[k]$, respectively. Since the number of footsteps up until a certain number of time samples cannot exceed that number of time samples: $(\mathbf{U}_b[k])_{i,j} = 0, \forall j > i$.

In most walking controllers based on MPC (Herd et al., 2010a; Maximo et al., 2016), footstep rotations are prescribed to avoid adding non-linearity to the problem. The work in Maximo and Afonso (2020) overcame this limitation by using binary variables, therefore allowing the robot to decide footstep rotation from a set of discrete values $\theta_f^p[k] \in \mathbb{R}^{N_r}$. Taking into account the decision variables presented in (11), the support foot rotation at footstep j is given by (13)

$$\theta_j[k] = \sum_{m=1}^{N_r} (\theta_f^p[k])_{j,m} \theta_{f,m}^p[k], \quad 1 \leq j \leq N_f, \quad (13)$$

whereas support foot rotation at time step i is computed as

$$\theta_{i,j}[k] = \sum_{m=1}^{N_r} (\theta_f^{aux}[k])_{i,m} \theta_{f,m}^p[k], \quad 1 \leq i \leq N. \quad (14)$$

We choose the elements of θ_f^p as

$$\theta_{f,m}^p = \theta_c + (m-1)T\dot{\theta}_r, \quad 1 \leq m \leq N_r, \quad (15)$$

where θ_c and $\dot{\theta}_r$ are the current support foot rotation and reference angular speed, respectively.

4.1.3. Cost function

We use a quadratic cost function that penalizes jerk usage, reference velocity tracking error, distance between the ZMP and the center of the support foot, and deviation from reference step duration and feet rotations:

$$\begin{aligned} J(\mathbf{u}[k]) = & \frac{\alpha}{2} \|\ddot{\mathbf{X}}[k]\|^2 + \frac{\beta}{2} \|\dot{\mathbf{X}}_r[k] - \dot{\mathbf{X}}[k]\|^2 + \frac{\gamma}{2} \|\mathbf{X}_f^{aux} - \mathbf{Z}_x[k]\|^2 \\ & + \frac{\alpha}{2} \|\ddot{\mathbf{Y}}[k]\|^2 + \frac{\beta}{2} \|\dot{\mathbf{Y}}_r[k] - \dot{\mathbf{Y}}[k]\|^2 + \frac{\gamma}{2} \|\mathbf{Y}_f^{aux} - \mathbf{Z}_y[k]\|^2 \\ & + \frac{\delta}{2} \sum_{j=1}^{N_f} \|S_{r,j}[k] - S_j[k]\|^2 + \frac{\xi}{2} \sum_{j=2}^{N_f} \|\theta_{r,j}[k] - \theta_j[k]\|^2, \end{aligned} \quad (16)$$

where $\dot{\mathbf{X}}_r[k]$ and $\dot{\mathbf{Y}}_r[k]$ represent vectors containing reference velocities along the prediction horizon; $S_{r,j}[k]$ and $\theta_{r,j}[k]$ are reference duration and rotation for the j -th footstep, respectively, which are defined in Section 4.1.4; $S_j[k]$ is the duration of the j -th footstep in time samples; and the non-negative weights $\alpha, \beta, \gamma, \delta$, and ξ define a compromise between the objectives. The ZMP reference is placed at the center of the support foot for stability reasons (Herd et al., 2010a; Maximo and Afonso, 2020). In this article, we consider the following reference velocity vectors: $\dot{\mathbf{X}}_r[k] = [\dot{x}_r]_N^T$ and $\dot{\mathbf{Y}}_r[k] = [\dot{y}_r]_N^T$.

A fixed reference step duration is used; e.g., a value known to minimize energy consumption may be chosen. To correctly impose reference step duration and footstep rotation in the optimization framework, we employed elaborate mechanisms based on constraints, which are explained in Sub Section 4.1.4.

4.1.4. Constraints

First, we remark that we use a common assumption in humanoid walking literature of considering that the double support phase endures one time step (Herd et al., 2010a; Bohórquez and Wieber, 2017, 2018). This abstains us from explicitly taking into account double support in our constraints. To ensure that the footsteps respect temporal constraints, the terms of $\mathbf{U}_b[k]$ are subject to the following constraints:

$$(\mathbf{U}_b[k])_{1,1} = 1, \quad (17)$$

$$\sum_{j=1}^{i+1} j (\mathbf{U}_b[k])_{i+1,j} - \sum_{j=1}^i j (\mathbf{U}_b[k])_{i,j} \geq 0, \quad 1 \leq i \leq N_f - 1, \quad (18)$$

$$\sum_{j=1}^{N_f} j (\mathbf{U}_b[k])_{i+1,j} - \sum_{j=1}^{N_f} j (\mathbf{U}_b[k])_{i,j} \geq 0, \quad N_f \leq i \leq N - 1, \quad (19)$$

$$\sum_{j=1}^{i+1} (o+j) (\mathbf{U}_b[k])_{i+1,j} - \sum_{j=1}^i (o+j) (\mathbf{U}_b[k])_{i,j} \leq 1, \quad 1 \leq i \leq N_f - 1, \quad (20)$$

$$\sum_{j=1}^{N_f} (o+j) (\mathbf{U}_b[k])_{i+1,j} - \sum_{j=1}^{N_f} (o+j) (\mathbf{U}_b[k])_{i,j} \leq 1, \quad N_f \leq i \leq N - 1, \quad (21)$$

where the offset o must be chosen according to:

$$o \geq N_f - 1, o \in \mathbb{Z}. \quad (22)$$

The first footstep is already decided and must be equal to the current one:

$$\begin{cases} x_{f,1}^b[k] = x_c \\ y_{f,1}^b[k] = y_c \end{cases}. \quad (23)$$

We consider that the current support foot rotation is at index 1 in the vector of possible rotations θ_f^b :

$$(\theta_f^b[k])_{1,1} = 1. \quad (24)$$

For stability, the ZMP position must be within the support polygon. Therefore, the ZMP constraints employ the support foot center at each time step i together with the respective chosen foot rotation:

$$(\theta_f^{aux}[k])_{i,m} \Rightarrow \mathbf{d}_s (\theta_{f,m}^p[k]) \begin{bmatrix} \mathbf{r}_{ZMP,x}[k+i|k] - x_{f,i}^{aux}[k] \\ \mathbf{r}_{ZMP,y}[k+i|k] - y_{f,i}^{aux}[k] \end{bmatrix} \leq \mathbf{s}, \quad 1 \leq i \leq N, \quad 1 \leq m \leq N_r, \quad (25)$$

where $\mathbf{d}_s (\theta_{f,m}^p[k])$ projects the ZMP error on the directions of the vectors normal to the edges of the support polygon when the foot is oriented with the m th rotation in θ_f^p and $\mathbf{s}$ contains the maximum deviation of the ZMP from the center of the support foot allowed by the support polygon in each direction.

The auxiliary variables $\mathbf{X}_f^{aux}[k]$, $\mathbf{Y}_f^{aux}[k]$, and θ_f^{aux} map support foot positions and rotations from footsteps to time steps:

$$(\mathbf{U}_b[k])_{i,j} \Rightarrow \begin{cases} x_{f,j}^b[k] = x_{f,i}^{aux}[k] & 1 \leq i \leq N \\ y_{f,j}^b[k] = y_{f,i}^{aux}[k] & 1 \leq j \leq \min(i, N_f) \end{cases}, \quad (26)$$

$$(\mathbf{U}_b[k])_{i,j} \Rightarrow \sum_{m=1}^{N_r} m (\theta_f^b[k])_{j,m} = \sum_{m=1}^{N_r} m (\theta_f^{aux}[k])_{i,m}, \quad 1 \leq i \leq N, \quad 1 \leq j \leq \min(i, N_f). \quad (27)$$

Velocity limits for the swing foot are imposed by

$$\mathbf{d}_v(\theta_c) \begin{bmatrix} x_{f,2}^b[k] - x_d[k] \\ y_{f,2}^b[k] - y_d[k] \end{bmatrix} \leq (S_1[k] - b_{ds}) T \mathbf{v}_{max}, \quad (28)$$

¹ $[a]_N = [a, \dots, a]^T$ represents a vector comprised by the scalar a repeated N times.

where $\mathbf{v}_{max} = [v_{x,max}, v_{x,max}, v_{y,max}, v_{y,max}]^T$ ($v_{x,max}$ and $v_{y,max}$ are maximum foot speeds in the x and y directions, respectively), $[x_a[k], y_a[k]]^T$ are the coordinates of the swing foot at time step k , b_{ds} is a boolean which is 1 if the robot is in double support (the robot is in double support in the first time step of a footstep) and 0 otherwise, and $\mathbf{d}_v(\theta)$ projects the acceleration vector along the four Cartesian movement directions (forward, backward, left, and right) when the foot is oriented with rotation θ . We also assume that the robot's torso has the same orientation as the current support foot, which is reasonable as an approximation.

For the footsteps other than the first, double support duration is accounted for by subtracting 1 from the time the swing foot is allowed to move:

$$\begin{aligned} (\theta_f^b[k])_{j,m} \wedge b_{s,j} \Rightarrow \mathbf{d}_d(\theta_{f,m}^p[k]) \begin{bmatrix} x_{f,j+1}[k] - x_{f,j-1}[k] \\ y_{f,j+1}[k] - y_{f,j-1}[k] \end{bmatrix} \leq (S_j[k] - 1) T \mathbf{v}_{max}, \\ 1 \leq m \leq N_r, 2 \leq j \leq N_f - 1. \end{aligned} \quad (29)$$

Notice that $b_{s,j}$ is used to relax this constraint if the j -th footstep is not completely within the prediction horizon. Constraining foot rotation is analogous:

$$-S_1[k] T \dot{\theta}_{max} \leq \theta_2[k] - \theta_a[k] \leq S_1[k] T \dot{\theta}_{max}, \quad (30)$$

$$b_{s,j} \Rightarrow -(S_j[k] - 1) T \dot{\theta}_{max} \leq \theta_{j+1}[k] - \theta_{j-1}[k] \leq (S_j[k] - 1) T \dot{\theta}_{max}, \quad (31)$$

$$2 \leq j \leq N_f - 1,$$

where $\dot{\theta}_{max}$ is the maximum rotational speed attained by the foot and $\theta_a[k]$ is the swing foot orientation. Moreover, acceleration limits are taken into account by the following inequalities:

$$\begin{aligned} (\theta_f^{aux}[k])_{i,m} \Rightarrow \mathbf{d}_a(\theta_{f,m}^p[k]) \begin{bmatrix} \ddot{x}[k+i-1][k] \\ \ddot{y}[k+i-1][k] \end{bmatrix} \leq \mathbf{a}_{max}, \\ 1 \leq i \leq N, 1 \leq m \leq N_r, \end{aligned} \quad (32)$$

in which $\mathbf{a}_{max} = [a_{x,max}, a_{x,max}, a_{y,max}, a_{y,max}]^T$ ($\mathbf{d}_a(\theta)$ is equal to $\mathbf{d}_v(\theta)$, and $a_{x,max}$ and $a_{y,max}$ are the limits of the CoM acceleration along the x and y directions, respectively).

We implement feet collision avoidance through the addition of the following linear constraints:

$$\begin{aligned} (\theta_f^b[k])_{j,m} \Rightarrow \\ (-1)^{j+b_{ls}-1} \left[-\sin(\theta_{f,m}^p[k]) \quad \cos(\theta_{f,m}^p[k]) \right] \begin{bmatrix} x_{f,j+1}^b[k] - x_{f,j}^b[k] \\ y_{f,j+1}^b[k] - y_{f,j}^b[k] \end{bmatrix} \leq -y_{sep}, \\ 1 \leq j \leq N_f - 1, 1 \leq m \leq N_r, \end{aligned} \quad (33)$$

where b_{ls} is a boolean which is 1 if the left foot is the current swing foot and 0 otherwise, y_{sep} is a minimum feet separation margin, and $\theta_{f,m}^p[k]$ is used to account for the foot rotation. Furthermore, to avoid the possibility of collision due to rotation, we impede the feet from pointing inward:

$$(-1)^{j+b_{ls}-1} (\theta_{j+1}[k] - \theta_j[k]) \leq 0, \quad 1 \leq j \leq N_f - 1. \quad (34)$$

To impose a reference step duration, an intricate mechanism was needed (Maximo and Afonso, 2020). Additional binary variables $b_{s,j}$ are defined as

$$b_{s,j} = \begin{cases} 1, & \text{use predetermined reference for footstep } j \\ 0, & \text{adapt footstep duration for footstep } j \end{cases}, \quad 1 \leq j \leq N_f. \quad (35)$$

The real variables $S_{r,j}$ represent the actual step durations and are obtained through the following constraints:

$$b_{s,j}[k] \Rightarrow S_{r,j}[k] = n_j, \quad 1 \leq j \leq N_f, \quad (36)$$

$$\neg b_{s,j}[k] \Rightarrow S_{r,j}[k] = N - \sigma_j, \quad 1 \leq j \leq N_f, \quad (37)$$

where σ_j represents the number of time steps in the previous footsteps and n_j is the predetermined duration for footstep j . We use n_j to discount time steps already being on the current step:

$$n_1 = \max(n_{ref} - k', 0), \quad (38)$$

$$n_j = n_{ref}, \quad 2 \leq j \leq N_f, \quad (39)$$

in which $n_{ref} \leq N$ is the desired step duration in steady-state and k' is the number of time samples that the robot has spent in the current step.

Moreover, the binary variables $b_{s,j}$ are also subject to

$$(N - n_j) b_{s,j} \geq N - \sigma_j - n_j, \quad 1 \leq j \leq N_f, \quad (40)$$

$$n_j b_{s,j} \leq N - \sigma_j, \quad 1 \leq j \leq N_f. \quad (41)$$

Each footstep must have only one rotation value:

$$\sum_{m=1}^{N_r} (\theta_f^b[k])_{j,m} = 1, \quad 1 \leq j \leq N_f, \quad (42)$$

$$\sum_{m=1}^{N_r} (\theta_f^{aux}[k])_{i,m} = 1, \quad 1 \leq i \leq N. \quad (43)$$

If a footstep is not within the prediction horizon, then we should not penalize its rotation tracking error. So, we make the reference equal to the value itself in this case:

$$\sum_{i=1}^N (\mathbf{U}_b)_{i,j} = 0 \Rightarrow \theta_{r,j}[k] = \theta_j[k], \quad 2 \leq j \leq N_f. \quad (44)$$

For valid footsteps, we use the following constraints to automatically compute reference rotation:

$$\theta_{r,1}[k] = \theta_c[k] \quad (45)$$

$$\sum_{i=1}^N (\mathbf{U}_b)_{i,j} \geq 1 \Rightarrow \theta_{r,j}[k] = \theta_{r,j-1}[k] + \Delta\theta_{r,j}[k], \quad 2 \leq j \leq N_f, \quad (46)$$

$$b_{s,2} \Rightarrow \Delta\theta_{r,2}[k] = \left(\frac{1 + (-1)^{b_{ls}+b_{ccw}}}{2} \right) (k' + S_1[k] + S_2[k]) T \dot{\theta}_r[k], \quad (47)$$

$$b_{s,j} \Rightarrow \Delta\theta_{r,j}[k] = \left(\frac{1 + (-1)^{j+b_{ls}+b_{ccw}}}{2} \right) (S_{j-1}[k] + S_j[k]) T \dot{\theta}_r[k], \quad 3 \leq j \leq N_f, \quad (48)$$

$$\neg b_{s,2} \Rightarrow \Delta\theta_{r,2}[k] = \left(\frac{1 + (-1)^{l+b_{ccw}}}{2} \right) (k' + S_1[k] + n_2) T \dot{\theta}_r[k], \quad (49)$$

$$\neg b_{s,j} \Rightarrow \Delta\theta_{r,j}[k] = \left(\frac{1 + (-1)^{j+b_{ls}+b_{ccw}}}{2} \right) (S_{j-1}[k] + n_j) T \dot{\theta}_r[k], \quad 3 \leq j \leq N_f, \quad (50)$$

where:

$$b_{ccw} = \begin{cases} 1, & \text{if } \dot{\theta}_r \geq 0 \\ 0, & \text{otherwise} \end{cases}. \quad (51)$$

i.e. boolean b_{ccw} is 1 if the robot is commanded to rotate counterclockwise and 0 otherwise.

To efficiently solve the MIQP defined in this subsection, we utilized the Gurobi optimization framework (Gurobi Optimization, Inc., 2015), which employs a branch-and-cut algorithm to perform the optimization.

4.2. Humanoid walking with imitation learning

The goal of the proposed neural network is to clone the behavior of the model predictive controller, as to be able to run the walking algorithm in real time on the embedded computer of a real robot. For that objective, the model must have access to the inner states of the robot, and predict how it should act at a given point in time. With that said, we can formulate a neural network that takes as inputs the reference velocity $\mathbf{v}_r = [\dot{x}_r, \dot{y}_r, \dot{\theta}_r]^T$ and the states of the robot at time step k , and outputs the decided jerks at time step k , the predicted swing foot pose at time step $k + 1$, and a boolean indicating if the robot is touching the ground at time step k . Mathematically, this can be represented as shown:

$$\text{Inputs} = \begin{bmatrix} \mathbf{x}[k] \\ \mathbf{y}[k] \\ x_a[k] \\ y_a[k] \\ \theta_a[k] \\ \mathbf{v}_r[k] \\ k'[k] \end{bmatrix}, \text{Outputs} = \begin{bmatrix} \ddot{\mathbf{x}}[k|k] \\ \ddot{\mathbf{y}}[k|k] \\ x_a[k+1|k] \\ y_a[k+1|k] \\ \theta_a[k+1|k] \\ b_{touch}[k+1|k] \end{bmatrix}, \quad (52)$$

where $\mathbf{x}[k] = [x[k], \dot{x}[k], \ddot{x}[k]]^T$ and $\mathbf{y}[k] = [y[k], \dot{y}[k], \ddot{y}[k]]^T$ are vectors containing the states of the robot's CoM (position, velocity, and acceleration), x_a , y_a and θ_a define the robot's swinging foot's position and orientation, and $k'[k]$ is a discrete variable informing how many time steps the robot has spent in the current footstep. As for the outputs, $b_{touch}[k+1|k]$ is a binary variable that says if the robot's swing foot should touch the ground at the next time step, and $\ddot{\mathbf{x}}[k|k]$ and $\ddot{\mathbf{y}}[k|k]$ are the jerks of the robot's CoM given the states at instant k , that will guide the ZMP to stay inside the support polygon, according to (5).

Although we do not have direct access to actuate in the jerk $\ddot{\mathbf{x}}[k|k]$ of the center of mass, we predict it as the output of the network instead of $\mathbf{x}[k]$ to conform with the walking algorithm proposed in [Maximo and Afonso \(2020\)](#), that uses it to derive the center of mass trajectory and to guide the ZMP throughout the walking. It is also important to notice that, despite the inputs $\theta_a[k]$ being originally discrete – see Section 4.1 – both the input $\theta_a[k]$ and the output $\theta_a[k+1|k]$ are treated as continuous values by the network, given that it is easier for an NN to deal with continuous values, and that the choice of it being treated as discrete values in the original formulation is solely to maintain linearity in the problem.

Given that the robot is symmetric with respect to its y -axis, i.e. there is no difference between its left and right sides, we do not have to inform the model which leg is the support foot (in order to decrease the dimensionality), so we pre-process the data, before inputting it to the algorithm, mirroring every input that has the left foot as support, to be as if they had the right foot as support, as described hereunder.

$$y[k] \leftarrow \begin{cases} y[k] & \text{if } b_{ls}[k] = 1 \\ -y[k] & \text{if } b_{ls}[k] = 0, \end{cases} \quad (53)$$

$$\dot{y}[k] \leftarrow \begin{cases} \dot{y}[k] & \text{if } b_{ls}[k] = 1 \\ -\dot{y}[k] & \text{if } b_{ls}[k] = 0, \end{cases} \quad (54)$$

$$\ddot{y}[k] \leftarrow \begin{cases} \ddot{y}[k] & \text{if } b_{ls}[k] = 1 \\ -\ddot{y}[k] & \text{if } b_{ls}[k] = 0, \end{cases} \quad (55)$$

$$\ddot{\mathbf{y}}[k|k] \leftarrow \begin{cases} \ddot{\mathbf{y}}[k|k] & \text{if } b_{ls}[k] = 1 \\ -\ddot{\mathbf{y}}[k|k] & \text{if } b_{ls}[k] = 0, \end{cases} \quad (56)$$

$$y_a[k] \leftarrow \begin{cases} y_a[k] & \text{if } b_{ls}[k] = 1 \\ -y_a[k] & \text{if } b_{ls}[k] = 0, \end{cases} \quad (57)$$

$$\theta_a[k] \leftarrow \begin{cases} \theta_a[k] & \text{if } b_{ls}[k] = 1 \\ -\theta_a[k] & \text{if } b_{ls}[k] = 0, \end{cases} \quad (58)$$

$$y_a[k+1|k] \leftarrow \begin{cases} y_a[k+1|k] & \text{if } b_{ls}[k] = 1 \\ -y_a[k+1|k] & \text{if } b_{ls}[k] = 0, \end{cases} \quad (59)$$

$$\theta_a[k+1|k] \leftarrow \begin{cases} \theta_a[k+1|k] & \text{if } b_{ls}[k] = 1 \\ -\theta_a[k+1|k] & \text{if } b_{ls}[k] = 0, \end{cases} \quad (60)$$

$$\dot{y}_r[k] \leftarrow \begin{cases} \dot{y}_r[k] & \text{if } b_{ls}[k] = 1 \\ -\dot{y}_r[k] & \text{if } b_{ls}[k] = 0, \end{cases} \quad (61)$$

$$\theta_r[k] \leftarrow \begin{cases} \theta_r[k] & \text{if } b_{ls}[k] = 1 \\ -\theta_r[k] & \text{if } b_{ls}[k] = 0, \end{cases} \quad (62)$$

where b_{ls} is a binary variable that indicates whether or not the robot is using its left foot as swinging foot. With this formulation, we reduce the dimensionality of the problem, whilst enforcing symmetry.

Furthermore, the frame of reference of the robot for the algorithm is its support foot, yet the simulations (including the one that generates the dataset – see Section 4.3) use an inertial frame of reference. For that reason, we also pre-process the data to translate and rotate with respect to x_c , y_c and θ_c , which are the positions and rotation of the robot's support foot.

$$\begin{bmatrix} x[k] \\ y[k] \\ 1 \end{bmatrix} \leftarrow \begin{bmatrix} \cos(\theta_c[k]) & \sin(\theta_c[k]) & -x_c[k] \\ -\sin(\theta_c[k]) & \cos(\theta_c[k]) & -y_c[k] \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x[k] \\ y[k] \\ 1 \end{bmatrix}, \quad (63)$$

$$\begin{bmatrix} x_a[k] \\ y_a[k] \\ 1 \end{bmatrix} \leftarrow \begin{bmatrix} \cos(\theta_c[k]) & \sin(\theta_c[k]) & -x_c[k] \\ -\sin(\theta_c[k]) & \cos(\theta_c[k]) & -y_c[k] \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_a[k] \\ y_a[k] \\ 1 \end{bmatrix}, \quad (64)$$

$$\begin{bmatrix} x_a[k+1|k] \\ y_a[k+1|k] \\ 1 \end{bmatrix} \leftarrow \begin{bmatrix} \cos(\theta_c[k]) & \sin(\theta_c[k]) & -x_c[k] \\ -\sin(\theta_c[k]) & \cos(\theta_c[k]) & -y_c[k] \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_a[k+1|k] \\ y_a[k+1|k] \\ 1 \end{bmatrix}, \quad (65)$$

$$\begin{bmatrix} \dot{x}[k] \\ \dot{y}[k] \end{bmatrix} \leftarrow \begin{bmatrix} \cos(\theta_c[k]) & \sin(\theta_c[k]) \\ -\sin(\theta_c[k]) & \cos(\theta_c[k]) \end{bmatrix} \begin{bmatrix} \dot{x}[k] \\ \dot{y}[k] \end{bmatrix}, \quad (66)$$

$$\begin{bmatrix} \ddot{x}[k] \\ \ddot{y}[k] \end{bmatrix} \leftarrow \begin{bmatrix} \cos(\theta_c[k]) & \sin(\theta_c[k]) \\ -\sin(\theta_c[k]) & \cos(\theta_c[k]) \end{bmatrix} \begin{bmatrix} \ddot{x}[k] \\ \ddot{y}[k] \end{bmatrix}, \quad (67)$$

$$\begin{bmatrix} \ddot{\mathbf{x}}[k|k] \\ \ddot{\mathbf{y}}[k|k] \end{bmatrix} \leftarrow \begin{bmatrix} \cos(\theta_c[k]) & \sin(\theta_c[k]) \\ -\sin(\theta_c[k]) & \cos(\theta_c[k]) \end{bmatrix} \begin{bmatrix} \ddot{\mathbf{x}}[k|k] \\ \ddot{\mathbf{y}}[k|k] \end{bmatrix}, \quad (68)$$

$$\begin{bmatrix} \dot{x}_r[k] \\ \dot{y}_r[k] \end{bmatrix} \leftarrow \begin{bmatrix} \cos(\theta_c[k]) & \sin(\theta_c[k]) \\ -\sin(\theta_c[k]) & \cos(\theta_c[k]) \end{bmatrix} \begin{bmatrix} \dot{x}_r[k] \\ \dot{y}_r[k] \end{bmatrix}, \quad (69)$$

$$\theta_a[k] \leftarrow \theta_a[k] - \theta_c[k], \quad (70)$$

$$\theta_a[k+1|k] \leftarrow \theta_a[k+1|k] - \theta_c[k]. \quad (71)$$

With that approach, the inputs to the model are always centered around the support foot, and always have the right foot as the support foot, without compromising the generalization of the model and reducing its dimensionality.

4.3. Dataset generation

To create a dataset with extensive variety that effectively represents all potential states in which the robot could find itself, we used a MATLAB[®] simulation, which models the robot walking using the original LIPM architecture (described in Section 4.1). The model operates in discrete time frames and is guided by (4) and (5) for determining the x and y positions. The duration of each time step is denoted as T_{sim} . Disturbances are represented as instantaneous changes in the CoM velocity components $\dot{x}$ and $\dot{y}$. State vector feedback is also assumed to be accessible. The key settings for this model, influenced by our custom-designed Chape humanoid robot, are outlined in Table 2 (refer to Fig. 10 for the robot).

Each run of the model yielded a trajectory where the robot followed a reference for 50 discrete time intervals. These trajectories were monitored to extract the input–output pairs for the dataset. Various dataset setups were tested to strike an optimal balance between diversity and

Table 2

Main parameters used in the simulation.

Description	Symbol	Value
Controller time step	T	0.06 s
Simulation	T_{sim}	0.06/18 s
CoM height	h	0.24 m
Maximum frontal speed	$v_{x_{max}}$	0.6 m/s
Maximum lateral speed	$v_{y_{max}}$	0.4 m/s
Maximum rotational speed	$v_{\theta_{max}}$	4 rad/s
Maximum frontal acceleration	$a_{x_{max}}$	2 m/s ²
Maximum lateral acceleration	$a_{y_{max}}$	2 m/s ²
Minimum lateral separation between the feet	y_{sep}	0.11 m
Foot length	l	0.055 m
Foot width	w	0.025 m
Prediction horizon length	N	10
Maximum number of footsteps in the prediction horizon	N_f	3
Number of possible rotations	N_r	11
Reference step duration	S_r	5 time steps
Double support duration	N_{ds}	1 time step

representability. While the robot should primarily walk stably at most times, the architecture is designed for robustness, capable of adapting to assorted disruptions to maintain stability. Hence, it is crucial to incorporate both disturbed and undisturbed trajectories in the dataset.

Given this context, the objective was to create a dataset that comprehensively captures the robot's possible states but remains focused on its primary function – multidirectional walking. The trajectories needed to show some level of unpredictability (due to disturbances) without being so random as to hinder learning. To meet this requirement, we constructed a dataset following a uniform distribution:

$$\begin{aligned} \dot{x}_r &\sim b_1 \cdot \mathcal{U}([-0.25, 0.25]) \text{ m/s}, \\ \dot{y}_r &\sim b_2 \cdot \mathcal{U}([-0.25, 0.25]) \text{ m/s}, \\ \dot{\theta}_r &\sim b_3 \cdot \mathcal{U}\left(\left[-\frac{\pi}{2}, \frac{\pi}{2}\right]\right) \text{ rad/s}, \quad b_i \in \{0, 1\}, i = 1, 2, 3, \end{aligned} \quad (72)$$

and with the following initial states

$$\begin{aligned} \theta_c &= \theta_a = 0, \\ x &= x_c = x_a = 0, \\ y &= y_c = y_{sep}, \\ y_a &= 0. \end{aligned} \quad (73)$$

These initial states make the robot initiate its trajectory in a one-foot support phase, with its CoM aligned over the supporting foot, poised and stable for the commencement of walking.

As indicated by Eq. (72), the reference speeds for a given trajectory are selected at random, and their direction is influenced by a binary factor (where $p(b_i = 0) = p(b_i = 1) = 0.5$). This approach is employed to create trajectories that include both unidirectional and multidirectional references, thereby enriching the dataset's diversity.

Additionally, there is a 50% probability that a given trajectory will experience an arbitrary disturbance to the robot's CoM velocity, which is uniformly distributed.

$$\begin{aligned} \dot{x}_{dist} &\sim \mathcal{U}([-0.3, 0.3]) \text{ m/s}, \\ \dot{y}_{dist} &\sim \mathcal{U}([-0.3, 0.3]) \text{ m/s}. \end{aligned} \quad (74)$$

Should a disturbance impact the trajectory, the simulation introduces random noise in both x and y velocities according to the specified distributions, occurring at an arbitrary time step, mimicking a push on the robot. The probability and magnitude of these disturbances were determined through experimentation to optimize dataset variability without generating outliers or unfeasible data. A sample is considered feasible if the MIQP optimization yields a solution enforcing the constraints, and unfeasible otherwise.

The finalized dataset comprises 3000 trajectories, each with 50 time steps, totaling 150,000 samples. Of these, 78.5% are feasible and can

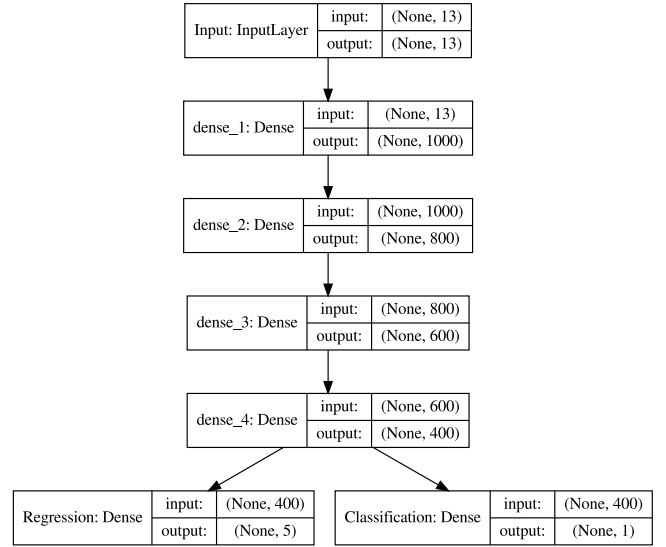


Fig. 2. Proposed model architecture taken from Keras. Each box is a layer of the network. Inside each box, the leftmost side is the layer's name, and the rightmost side shows the dimension, divided into (batch size, dimension) of inputs and outputs. The arrows indicate the feedforward direction of the network.

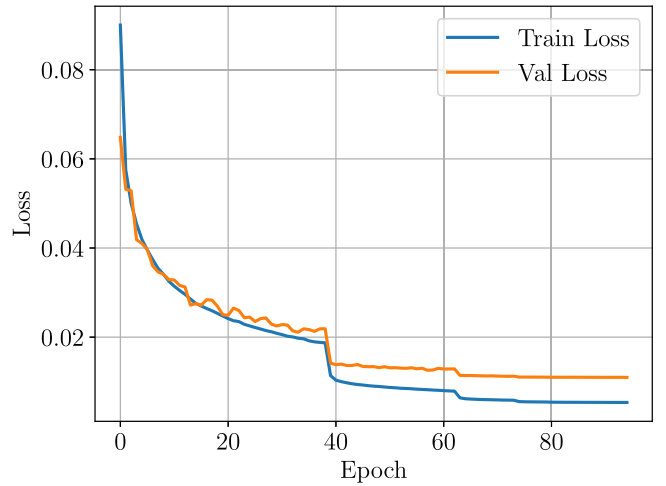


Fig. 3. Training and validation learning curves.

be used for training or testing. Unfeasible data are excluded since the simulation stops if the MIQP fails at any time step. The high number of unfeasible data stems from our aim to create a highly variable dataset, sometimes resulting in overly challenging conditions for the robot.

The dataset size was determined based on performance improvements plateauing at around 3000 trajectories. The dataset is then partitioned into 90% (105,975 samples) for training and 10% (11,750 samples) for validation.

Lastly, prior to model training, the dataset undergoes scaling to mitigate gradient-related issues. Given that the classification output b_{touch} is binary and the input variable k' is categorical, they are not scaled. All other variables are standardized to have a mean of 0 and a standard deviation of 1.

4.4. Proposed model

For replicating the gait behavior generated by the model predictive controller, we designed a two-headed deep fully connected

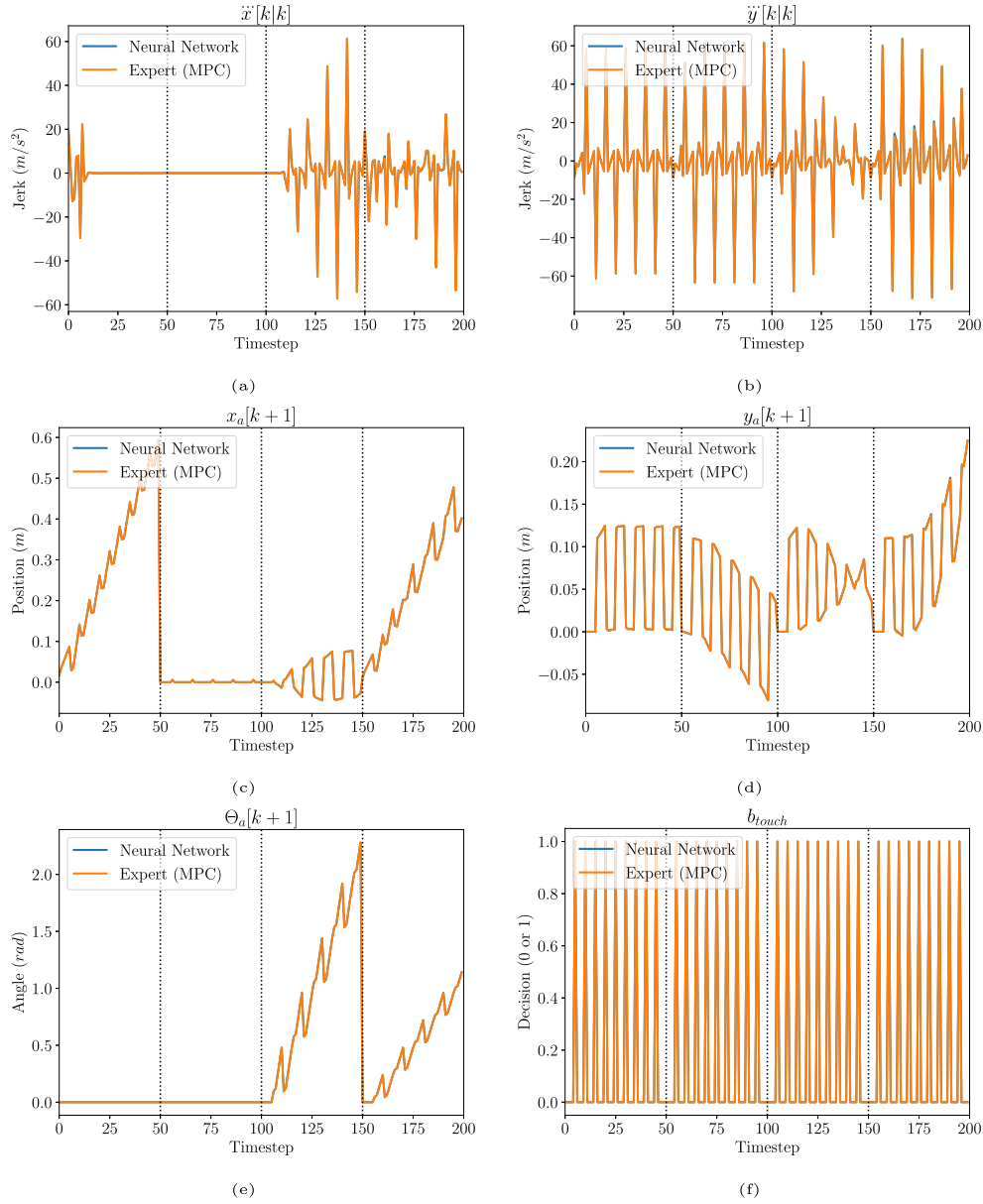


Fig. 4. Comparison between real and predicted values of the four trajectories chosen to test the model. (a) and (b) present the jerk of the center of mass in x and y , respectively; (c), (d) and (e) show the next position of the swinging foot in x , y and θ , respectively; (f) shows the duration of the step. Each dotted line indicates the end of one of the trajectories in Table 3 and the beginning of the subsequent one.

neural network, capable of both regression and classification tasks. The network takes 10 robot states and 3 directional velocity references at a specific time point, and predicts the swinging foot and center of mass positions for ZMP balancing on the regression side, while determining if the robot should make ground contact at the end of the current time step on the classification side.

Hyperparameters such as the number of layers, number of neurons per layer, and learning rate were fine-tuned through a blend of manual experimentation and an optimization technique rooted in the Tree-structured Parzen Estimator (TPE) algorithm (Zhao and Li, 2018), facilitated by the Optuna framework (Akiba et al., 2019). Multiple loss function configurations for both classification and regression tasks were explored to achieve optimal performance. The selected hyperparameter mix represents a local minimum in the optimization landscape. The network features 4 hidden layers with neuron counts of 1000, 800, 600, and 400, in that sequence. Each layer employs a rectified linear unit (ReLU) activation function (Nair and Hinton, 2010). The regression output layer uses a linear activation function, while the classification

output layer employs a sigmoid function, aligning with the binary nature of the target b_{touch} . The architecture of the model is illustrated in Fig. 2.

Given the network's relatively low number of trainable parameters and the substantial diversity and size of the dataset, we found that regularization techniques were unnecessary. The model demonstrated good generalization post-training, as detailed in Section 5.1. To balance the error contributions from both the regression and classification tasks, the classification loss was scaled by a factor of 0.15.

For the loss metrics, we employed Mean Absolute Error (MAE) for both the regression and classification outputs. Hyperparameter tuning revealed that MAE outperformed other regression loss functions such as mean squared error. For the classification task, MAE's performance was comparable to traditional classification losses such as binary cross-entropy. Given the convenience of numerical output for subsequent model applications, we opted for MAE for classification as well. A related discussion on using regression loss for classification tasks can be found in Redmon et al. (2016).

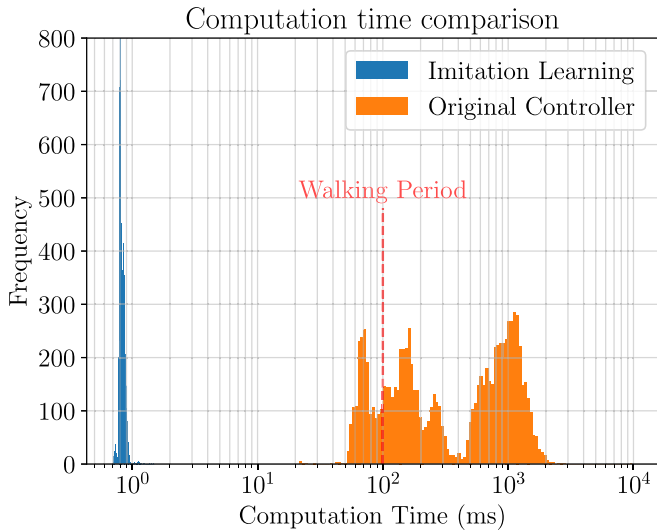


Fig. 5. Histograms of the computation times of both the neural network and the MPC-MIQP.

The Adam optimizer (Kingma and Ba, 2014) was chosen for its reputation as a state-of-the-art optimizer for DNNs, surpassing other options such as Adagrad and RMSprop (Zaheer and Shaziya, 2019). All Adam parameters were initialized to default settings, except for the learning rate, which was fine-tuned and adjusted according to a scheduler, as elaborated in Section 5.1.

5. Results

In this section, we train the model using a traditional supervised learning algorithm (Goodfellow et al., 2016) and the Tensorflow framework (Abadi et al., 2015), and compare the NN's output with the MPC's data, which utilizes an MIQP, using validation data to gauge performance. We also measure computation time to assess the advantages of deploying the NN over the MPC-MIQP solution. Subsequently, the NN is employed to drive the robot in MATLAB® simulations based on the LIPM, in Gazebo simulations featuring the robot's full-body model, and then integrated into the actual robot hardware. Each set of results is elaborated upon in its dedicated subsection. The parameters of the MPC were chosen to suit our real robot Chape. Video links showcasing the IL controller in full-body Gazebo simulations and on the real robot Chape are available at <https://youtu.be/-6eTjn0Jq1k>, https://youtu.be/i5i3zZ_xz7Q, and <https://youtu.be/uNj8J43uefo>.

5.1. Neural network

The designed network underwent training for a maximum of 200 epochs. Callbacks were implemented to decrease the learning rate by a factor of 5 whenever the validation loss failed to improve over a span of 5 epochs and to halt training if no improvement was observed for 10 epochs. The initial learning rate was set at 2.5×10^{-4} , and in the most successful run, the model completed 95 epochs. The learning trajectories are illustrated in Fig. 3.

Notice that the model did not overfit, as both training and validation losses plateaued by the end of the training process. Additionally, the strategy of reducing the learning rate upon reaching plateaus proved highly effective, as the model experienced notable performance gains at epochs 39 and 63 due to the learning rate adjustments.

The model was then tested on four different trajectories, and its predictions were compared to the original labels. The references of each trajectory can be found in Table 3, and the results of each test can be seen in Fig. 4. We can see in the figures that the model was able to

Table 3

Test trajectories' reference.

Trajectory	$\dot{x}$	$\dot{y}$	$\dot{\theta}$
1	0.2	0.0	0.0
2	0.0	-0.2	0.0
3	0.0	0.0	0.8
4	0.15	-0.1	0.4

correctly track the reference with minimal error in all the outputs. This shows that, regardless of the reference or the combination of directions the robot was ordered to walk, it could consistently perform the tasks following closely the behavior of the expert.

5.2. Computation time

A key advantage of imitation learning is its capacity to emulate expert performance but at a substantially reduced computational time. While the MPC ensures stable walking, its dependence on iterative optimization methods makes its computation time both resource-intensive and variable, as the MIQP may necessitate an unpredictable number of branching steps to find the optimal solution. Conversely, the neural network offers a stable inference time with minimal fluctuation and can execute in a fraction of the time required by MIQP.

To evaluate computational efficiency, we executed the neural network on the real robot Chape 9496 times. Due to implementation constraints, the MIQP was run a similar number of times on a desktop computer – see Table 4 – which is considerably more powerful than the robot's onboard computer. Fig. 5 displays the histogram of computation times, revealing that the neural network vastly outperformed the MIQP. Specifically, 99.52% of the neural network's instances were completed in under 1 ms, whereas the MIQP managed to keep only 18% of its instances under 100 ms, making it unsuitable for real-time applications.

Further, Fig. 5 and Table 4 show that the NN's time variance is extremely low, indicating its reliable performance within allocated time frames in higher-level algorithms. In contrast, MIQP's times ranged from around 20 ms to nearly 10,000 ms, making it susceptible to time overruns.

In summary, the NN's primary edge over the predictive controller is its computation time: it operated, on average, 650 times faster than the MIQP, with minimal variance. Any peak times are likely outliers attributable to other jobs running in the processor, rather than the network itself.

5.3. LIPM simulation

To extend the comparison between the MIQP and the IL controllers beyond database metrics to continuous walking, we employed the MATLAB® simulation originally used for trajectory generation — see Section 4.3. In this simulation, the robot walks for 50 time steps, following a given reference and potentially encountering disturbances, while we monitor all relevant variables like position, velocity, and jerks. This side-by-side evaluation offers a thorough examination of performance under controlled disturbances, allowing us to assess if the IL controller maintains the level of robustness exhibited by the MPC-MIQP controller.

5.3.1. Walking forward

Initially, we simulated the robot moving forward at a constant speed of 0.2 m/s without any disturbances, using both the MIQP and the IL controllers. As shown in Fig. 6, the NN closely mimics the MPC-MIQP in this straightforward, undisturbed scenario, with errors that are virtually indiscernible at the presented scale. Minor deviations are only noticeable in Figs. 6(c) and (d), where the NN takes slightly longer to stabilize the x velocity. Aside from this, the IL controller demonstrated

Table 4

Computation time comparison. std indicates the standard deviation of the times. Peak indicates the higher value encountered in the experiment.

Method	CPU Specs	Average Time (ms)	std (ms)	Peak (ms)
NN	i5-4250U @ 3.60 GHz	0.83	0.04	1.48
MPC-MIQP	i5-1035G1 @ 2.60 GHz	539.63	482.57	9818.35

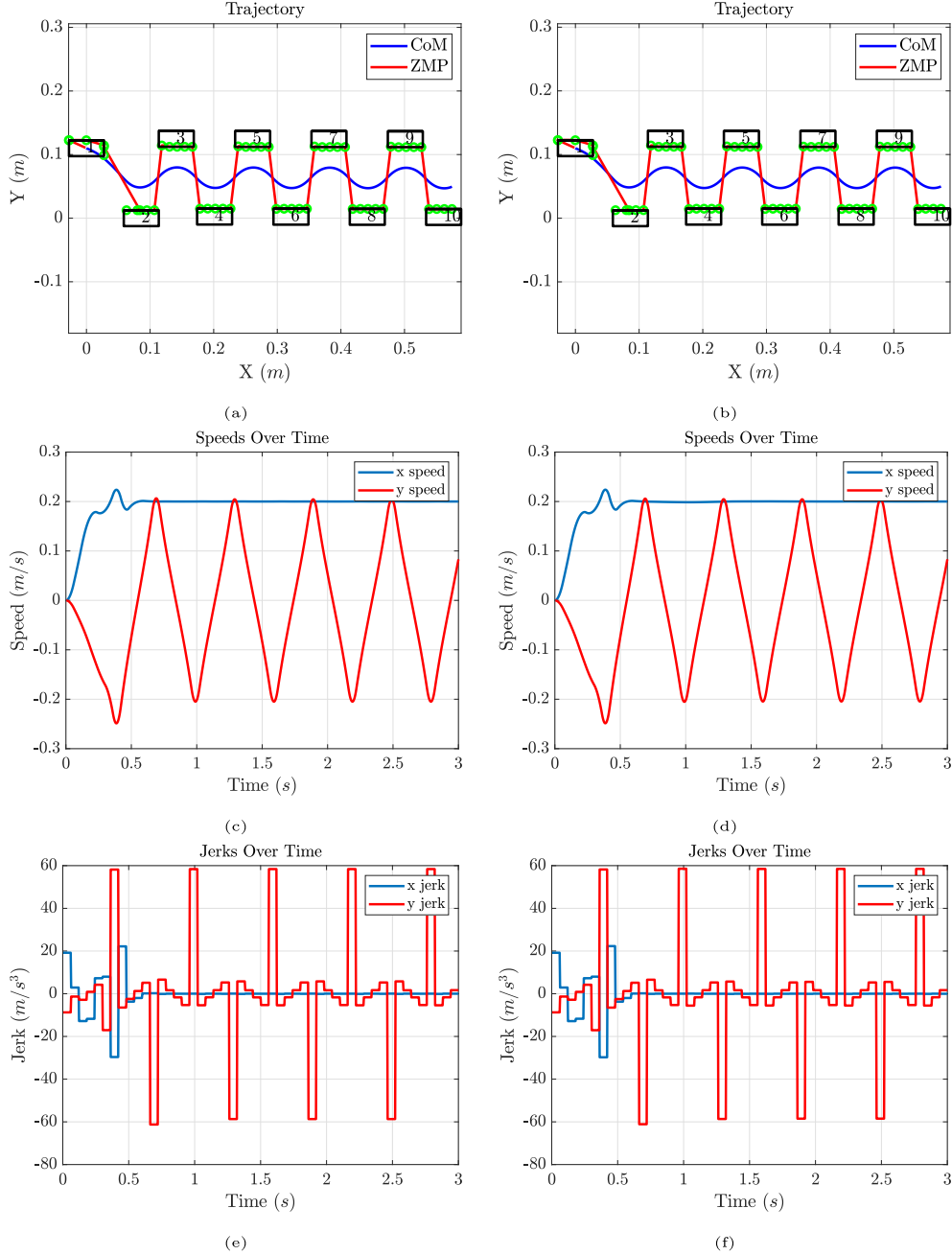


Fig. 6. LIPM simulation walking forward with a reference velocity of 0.2 m/s. The left side figures are from the MIQP controller, while the right side ones are from the IL controller. (a) and (b) show the robot's trajectory — the green circles show the discrete ZMP placements and the numbers inside the rectangles indicate the order of the footsteps; (c) and (d) show the x and y speeds over time; and (e) and (f) show the x and y jerks over time.

a strong ability to replicate the behavior of the MIQP controller in a basic but commonly encountered task.

5.3.2. Rotating

Subsequently, the simulation was configured to have the robot rotate around its axis at a constant angular velocity of 1 rad/s, again without any disturbances. The trajectories of both the IL and MIQP

controllers are virtually indistinguishable in Fig. 7. The IL controller generated a gait that kept the ZMP within the support polygon at every time step and successfully rotated around its axis, mirroring the expert's behavior.

Both the initial and subsequent experiments using the LIPM simulator yielded results that demonstrate the NN's capability to accurately emulate the MPC-MIQP in tasks requiring adherence to a reference

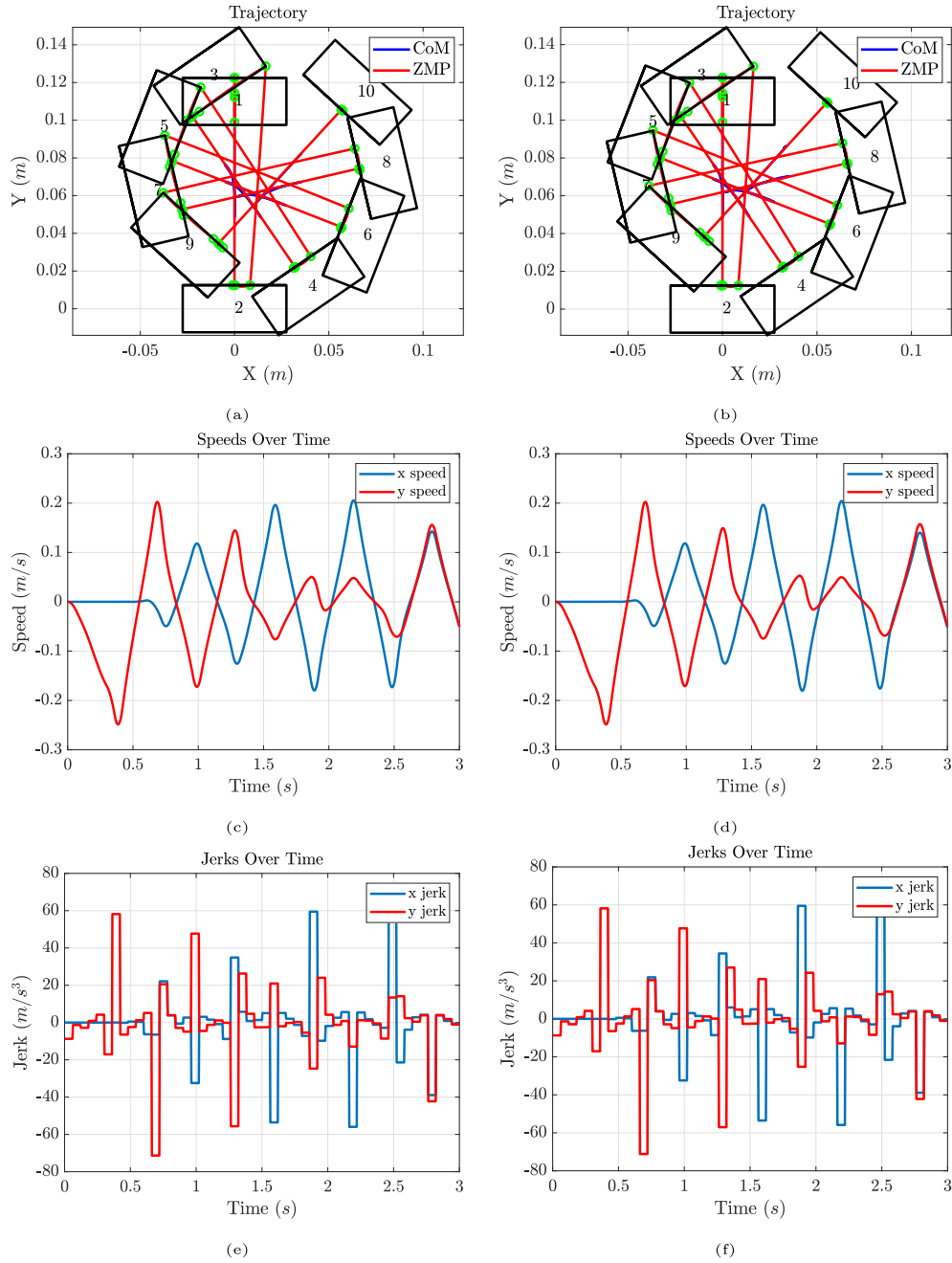


Fig. 7. LIPM simulation rotating around its axis with a reference velocity of 1 rad/s. The left side figures are from the MIQP controller, while the right side ones are from the IL controller. (a) and (b) show the robot's trajectory — the green circles show the discrete ZMP placements and the numbers inside the rectangles indicate the order of the footsteps; (c) and (d) show the x and y speeds over time; and (e) and (f) show the x and y jerks over time.

velocity, whether linear or angular. These findings showcase the IL controller's ability to mimic the MPC-MIQP's closed-loop behavior for the robot's most frequent task: walking according to a given reference.

5.3.3. Walking forward with backward push

Thus far, we have evaluated the IL controller's proficiency in tracking a reference velocity. However, a key strength of the original MIQP formulation lies in its resistance and adaptability to disturbances. To assess this aspect, the model was subjected to the same conditions as the first experiment — a constant reference velocity of 0.2 m/s along the x axis — but with an added disturbance of -0.2 m/s applied to its center of mass in the x direction at $t = 1.23$ s. The results of this test are presented in Fig. 8.

In this scenario, the differences between the two models become more apparent. In Figs. 8 (a) and (b), it is evident that while the robot covers roughly the same distance in both simulations and the positions of its support feet are nearly identical in each iteration, the ZMP on the fifth footstep — one step after being pushed — differs between the original and the proposed models. The neural network situates the ZMP nearer to the center of the support polygon, and the support foot is closer to the fourth footstep in the y direction.

In Figs. 8 (c) and (d), similar to the first experiment, the IL controller takes a longer time to stabilize the x velocity compared to the MPC, especially after experiencing the push. The backward push also causes a slight divergence in the y velocity, but it quickly returns to a pattern similar to the expert's.

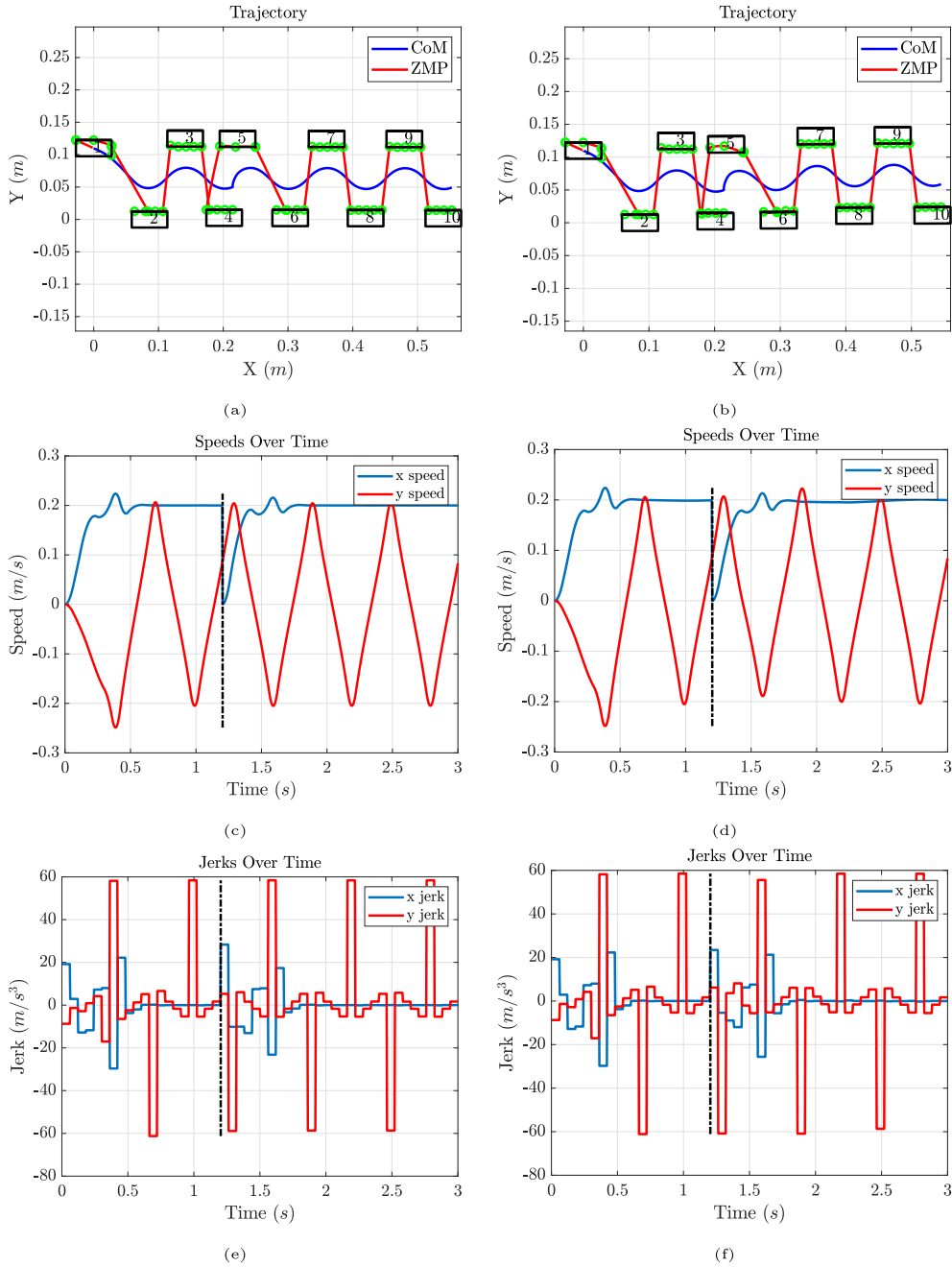


Fig. 8. LIPM simulation walking forward with a reference velocity of 0.2 m/s receiving a disturbance of -0.2 m/s in the x direction at $t = 1.23$ s. The left side figures are from the MIQP controller, while the right side ones are from the IL controller. (a) and (b) show the robot's trajectory — the green circles show the discrete ZMP placements and the numbers inside the rectangles indicate the order of the footsteps; (c) and (d) show the x and y speeds over time; and (e) and (f) show the x and y jerks over time.

Figs. 8 (e) and (f) further reveal some divergence in behavior between the IL and MIQP controllers, particularly following the disturbance at $t = 1.23$ s. However, this seems to be a transient state, as no discernible differences are observed from the next footstep onward.

Despite these observed differences – expected when dealing with new, unseen data samples – the robot still successfully completed the task and recovered smoothly from the disturbance, resuming the reference trajectory in the same number of time steps as the original controller.

5.3.4. Walking forward with side push

Lastly, we evaluated the robot's response to a side push of -0.15 m/s while walking forward with the same reference as the previous experiment, again in the LIPM simulation.

Much like the previous experiment, Figs. 9 (a) and (b) reveal a minor discrepancy in ZMP positioning immediately following the lateral disturbance. Nonetheless, the robot successfully regained control of its trajectory and recovered from the disturbance. Additionally, in both the left and right images, we observe that the robot extended its step duration at the fourth step – evidenced by six ZMP placements within the footstep as opposed to the default five – to counteract the disturbance.

In Figs. 9 (c) and (d), while both simulations could follow the y speed reference, especially after experiencing a side push, the NN faced greater difficulty in tracking the x speed post-disturbance compared to the MPC-MIQP, with a minor error persisting for an additional step.

In summary, the NN demonstrated its capability to walk at the desired reference speed and withstand disturbances effectively, as evidenced by the four experiments conducted. This highlights its

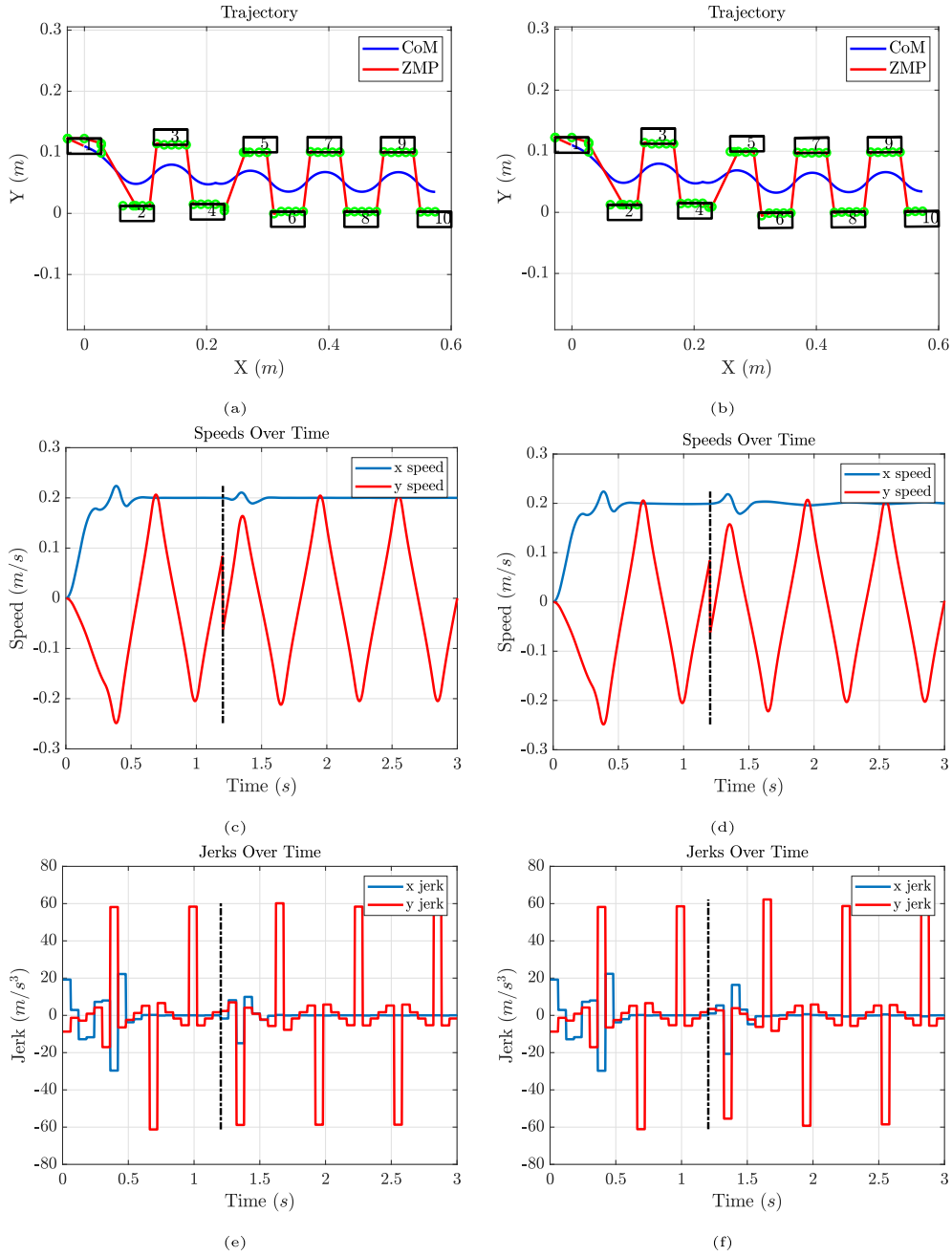


Fig. 9. LIPM simulation walking forward with a reference velocity of 0.2 m/s receiving a disturbance of -0.15 m/s in the y direction at $t = 1.23$ s. The left side figures are from the MIQP controller, while the right side ones are from the IL controller. (a) and (b) show the robot's trajectory — the green circles show the discrete ZMP placements and the numbers inside the rectangles indicate the order of the footsteps; (c) and (d) show the x and y speeds over time; and (e) and (f) show the x and y jerks over time.

proficiency in emulating the behavior of the original MIQP controller and its robust generalization ability, managing to withstand disturbances with minimal error relative to the MPC-MIQP — one of the key features of the original model.

5.4. Full-body simulation

We further assessed the robot's performance using our IL controller within a high-fidelity simulated environment that accurately captures the robot's physics and interactions with its surroundings. This simulation faithfully replicates the robot's body structure, joint mechanics, and ground contact forces. We also implemented gravity compensation and torso inclination adjustments to bridge the dynamic discrepancies between the full-body dynamics and the LIPM. The distinguishing

feature of the full-body simulation is that it allows for the introduction of disturbances with known magnitudes and durations, providing a systematic and reproducible framework for evaluating the disturbance tolerance of the proposed IL controller in realistic conditions. To achieve this, we employed the Gazebo simulator, incorporating our high-fidelity model of Chape. Testing the IL controller in such a realistic simulation environment is crucial for assessing the stability of the walking gait generated by the model, especially when subject to the limitations imposed by the physics, LIPM approximations, and joint dynamics.

5.4.1. Walking forward

The initial Gazebo simulation was designed to have the robot move forward at a constant velocity of 0.2 m/s. We monitored its positions over the course of the simulation, as depicted in Fig. 11.

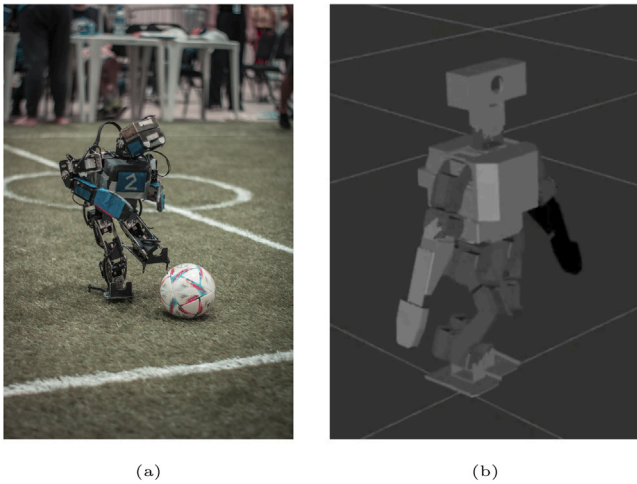


Fig. 10. The humanoid robot Chape and its high-fidelity Gazebo representation side by side.

From Fig. 11(b), it is evident that the IL controller was capable of generating a gait that followed the reference, despite some fluctuations caused by tracking errors from the robot's low-level controllers. Additionally, Fig. 11(a) displays a behavior closely mirroring that observed in the LIPM simulations (refer to Section 5.3.1). This not only attests to the reliability of the simulator but also confirms that the robot could maintain stable walking under the influence of realistic physics and tracking errors, all while utilizing the IL controller. Moreover, Figs. 11(c) and (d) indicate that the robot experienced minimal drift along the y axis, showing its ability to walk in a straight line.

5.4.2. Walking forward with backward push

In the second simulation setup, the robot was subjected to the same task — moving forward at a speed of 0.2 m/s. However, this time it encountered a backward push, introducing a disturbance of -0.2 m/s in the x direction at the 2-second mark.

In Figs. 12 (a) and (d), the robot's response to the backward push is evident. It modified its foot positions to regain stability post-disturbance, ensuring the ZMP remained within the support polygon. Additionally, Fig. 12(b) reveals that at the 2-second mark, immediately following the disturbance to its CoM velocity, the predicted x speed drops to zero — consistent with a disturbance of -0.2 m/s while walking at 0.2 m/s — and then returns to the reference in subsequent steps. Lastly, Fig. 12(c) shows that the backward push in the x direction had minimal impact on the robot's y direction, allowing it to continue walking in a straight line as intended.

From both high-fidelity simulations, it is evident that the proposed IL controller is able to withstand disturbances under realistic physics, effectively emulating the behavior of the MPC-MIQP and ensuring task stability throughout.

5.5. Disturbance margins

In Section 5.3, we demonstrated that the IL controller could effectively emulate the MIQP's behavior and resist disturbances. However, it is crucial to evaluate the upper limits of disturbances that both controllers can withstand. To this end, we conducted simulations using the LIPM model, as described in Section 5.3. In these tests, the robot was instructed to move forward at a reference velocity of 0.1 m/s, and a disturbance was introduced at the onset of its fifth step. The amplitude of the disturbance ranged from 0 to 0.4 m/s in increments of 0.02 m/s, while the disturbance's direction varied from 0 to 180 degrees in 30-degree steps.

It is important to note that the MPC-MIQP formulation takes into account the physical limitations of the robot, such as the size of the foot, which the IL controller does not explicitly consider. As a result, a direct comparison between the two controllers may not be appropriate, as a very small deviation of the ZMP could lead to a failure in the IL controller, even though it may be just outside the support polygon, with negligible errors. To address this issue, we enlarged the robot's foot by 2.5 mm in each boundary and considered the robot stable when the ZMP landed inside the extended support polygon. By doing so, we ensured that the evaluation of the IL controller was fair and accounted for its inherent limitations. We considered the robot to have withstood a disturbance if it did not lose stability upon being pushed, and was able to recover and continue following the reference velocity afterward.

Fig. 13 illustrates the combinations of disturbance amplitude and direction the MIQP and IL controllers could withstand. The IL controller largely matched the MIQP's performance, with a slight difference at the 60-degree angle, where the IL controller could resist up to 0.16 m/s, compared to the MIQP's 0.18 m/s.

We must take into consideration that, even though it theoretically failed to resist the impulse, i.e. it failed to keep the ZMP inside the support polygon at all times, it does not mean the robot will fall — see Section 3.1. To assess the disturbance margins the IL controller can withstand in a more realistic scenario, we repeated the previous experiment, but this time on Gazebo with the full-body simulation of Section 5.4.

We see in Fig. 14 that the IL controller could resist disturbances of amplitude equal to or greater than the MPC-MIQP, outperforming the original MPC. This comes from the fact that when the MPC-MIQP fails to keep the ZMP inside the support polygon, it does not return any solution, as no infeasibility treatment was added, and the robot falls. On the other hand, the IL controller can return a solution that does not contain the ZMP inside the support polygon for a certain period of time, but this does not lead to instability, and it successfully adapts its gait and returns to follow the reference.

These results show a good generalization power of the NN as it manages to output a configuration where the robot successfully executes the task even in situations where the expert does not return a solution. Furthermore, it shows that our choice of extending the robot's foot did not compromise the first experiments, as even with the original foot size the robot managed to successfully overcome the disturbances, indicating that our experimental setup was suitable to test the controllers under a range of disturbance conditions.

Additionally, since the IL model is data-driven, its stability is evaluated empirically (Precup et al., 2021) rather than through direct model analysis. Here, stability is demonstrated by rigorously testing the IL controller in extreme conditions, and ensuring it maintains a stable response in challenging scenarios.

5.6. Real robot

In a real-world setting, we evaluated the IL controller on the actual robot, Chape. A series of experiments were conducted to assess its ability to follow a reference velocity and maintain stable motion on a synthetic grass field. To monitor its position and orientation over time, a camera was mounted above the field.

5.6.1. Walking forward

The first experiment involved instructing the robot to move forward at a constant velocity of 0.15 m/s along the x -axis.

We can see from Fig. 15(a) that the robot maintained a constant velocity, and that it follows the reference of 0.15 m/s, given that it traveled close to 1.5 m in about 10 s. It is also possible to interpret, from Fig. 15(c), that the orientation of the robot stood steady around the origin, only oscillating a little due to the robot's slipping during the change of support foot. In that figure it is also possible to see that the

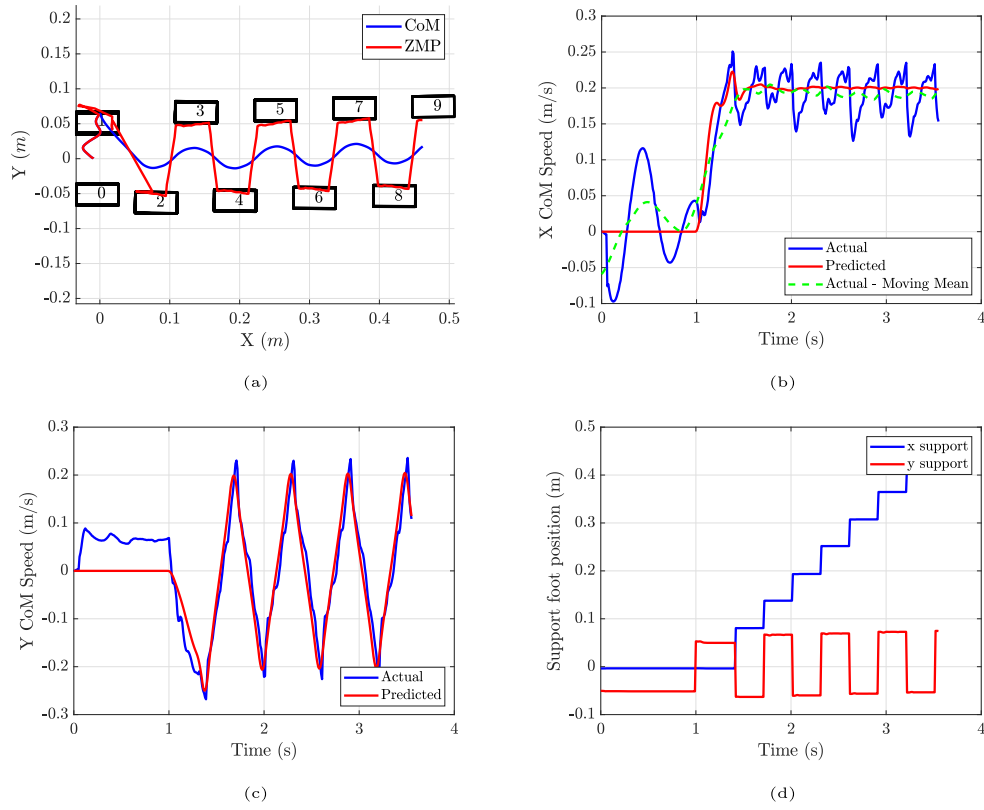


Fig. 11. Full-body simulation walking forward with a reference velocity of 0.2 m/s in Gazebo. (a) shows the gait generated by the network; (b) and (c) show CoM speed in the x and y directions, respectively — the blue curve is the actual speed, tracked directly from Gazebo and the red curve is the speed the robot would have if the model predictions would have been perfectly followed; (d) shows the support foot positions as a function of time.

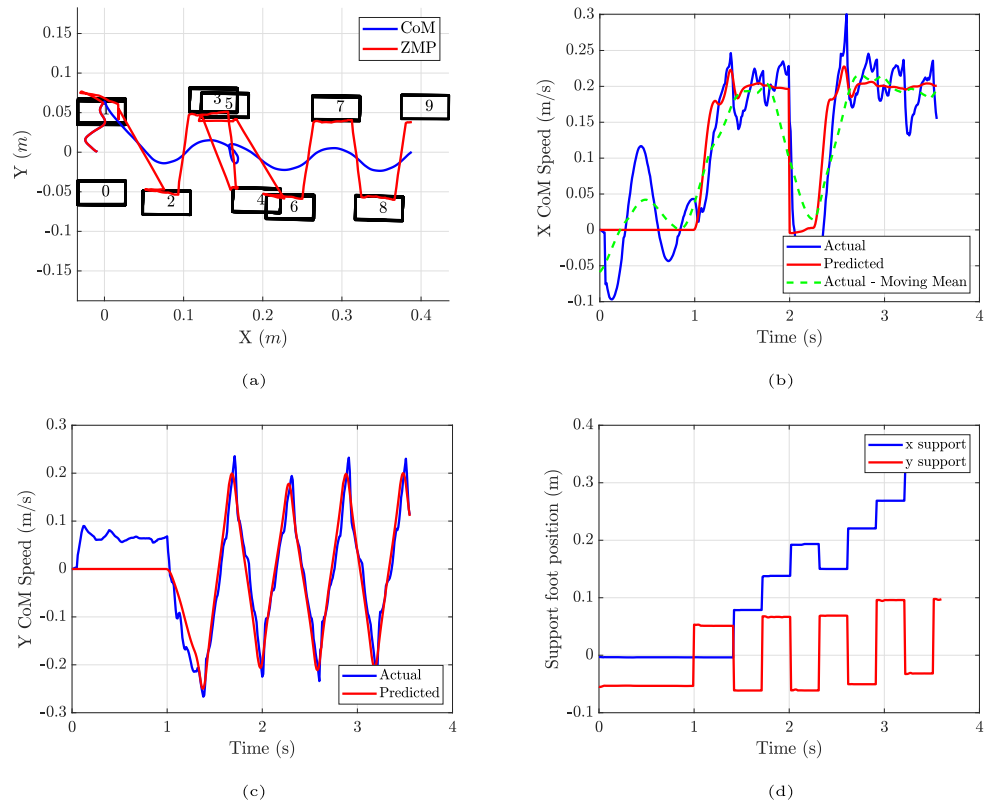


Fig. 12. Full-body simulation walking forward with a reference velocity of 0.2 m/s and with a disturbance of -0.2 m/s in x at $t = 2$ s in Gazebo. (a) shows the gait generated by the network; (b) and (c) show CoM speed in the x and y directions, respectively — the blue curve is the actual speed, tracked directly from Gazebo and the red curve is the speed the robot would have if the model predictions would have been perfectly followed; (d) shows the support foot positions as a function of time.

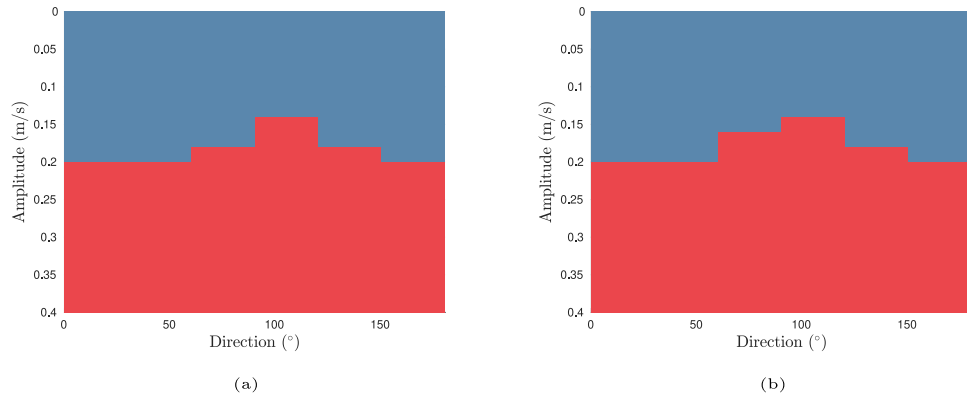


Fig. 13. Disturbance margins in the LIPM simulation. (a) shows the original MIQP controller margins; (b) shows the IL controller margins. The robot kept balance in the blue regions, while it failed to withstand the disturbance in the red regions.

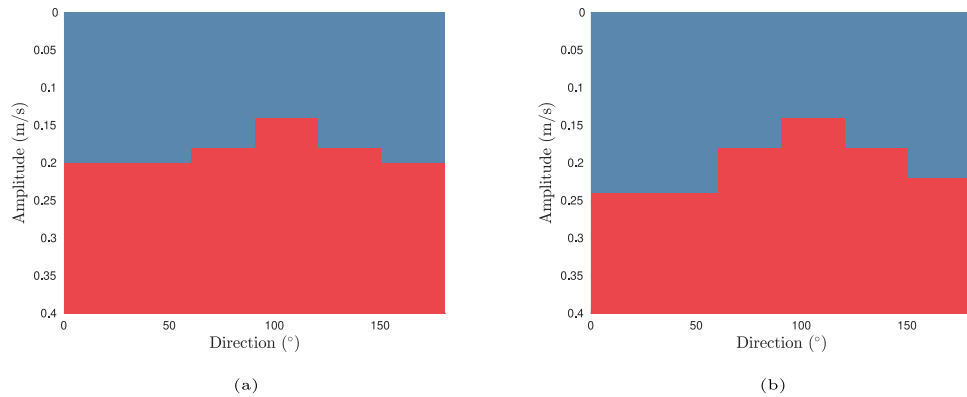


Fig. 14. Disturbance margins in the full-body realistic simulation. (a) shows the original MIQP controller margins; (b) shows the IL controller margins. The robot kept balance in the blue regions, while it failed to withstand the disturbance in the red regions.

robot initiated the experiment rotated a bit to the right (with its orientation not centered around zero), which explains why, in Fig. 15(b), the y position of the robot decreases over time. This has nothing to do with the proposed controller, but only with the error of initial condition inherent to real-life experiments. In that same Fig. 15(b), we can see the left and right tilting of the robot's CoM, characteristic of its change of support foot. We emphasize that the MIQP controller used to train the IL controller does not treat these deviations.

5.6.2. Changing reference

The second experiment with the real robot consisted of commanding it to walk forward for a fixed duration, and then make a complete turn of π rad.

From this experiment, it is easy to understand the behavior of the θ curve, in Fig. 16(c). It starts around zero (again with a small initial error) and stays still until the robot begins rotating, at around 2.3 s. From then on, it is possible to see that the robot completed an entire turn, rotating approximately π rad.

In Fig. 16(a), we can see that Chape starts with a constant velocity in the x direction, then stops moving forward, and, during the second phase of the experiment – the turning – its x position is also affected by the rotation of the robot, given that it no longer affects only the y axis. Furthermore, Fig. 16(b) shows, that, initially, the behavior on the y axis was very similar to that of the first experiment, but, once the robot started rotating, it was driven slightly to the left. This was due to the sudden change in reference, considering that the robot was not given instructions to slowly reduce its x velocity and then turn, but rather immediately change its reference. In view of its inertia and actuator limitations, in the first few seconds of the rotation, there is a composition of movements.

Moreover, Fig. 17 shows snapshots of a gait produced by the IL controller on the Chape robot from three views: (a) top view, (b) front view, (c) side view. These snapshots were produced by capturing a video of the robot walking with a reference in x of 0.15 m/s, and, although each of those views shows a similar experiment, they represent different instances of it.

6. Conclusions

In this paper, we presented a neural network designed to emulate the adaptive gait behavior produced by the model predictive controller outlined in Maximo and Afonso (2020). This controller is capable of adjusting footstep positions, rotations, and durations in response to disturbances.

Our findings indicate that the devised IL controller successfully learned the gait patterns generated by the MIQP controller. The model also demonstrated robust generalization when subjected to velocity disturbances, thereby providing the robot with push recovery features, a key attribute of the original controller. Additionally, the IL controller could be integrated into the actual robot and operated in real-time, overcoming the computational limitations that hindered the real-time application of the MPC-MIQP.

It follows that the IL controller effectively emulated the MPC-MIQP's behavior, sacrificing minimal accuracy and disturbance tolerance while achieving a 650-fold decrease in execution speed. Given the model's strong generalization capabilities, it suggests that the application of imitation learning to humanoid robot walking could be extended to include other expert controllers, thereby accelerating the development of real-time embedded robotic systems.

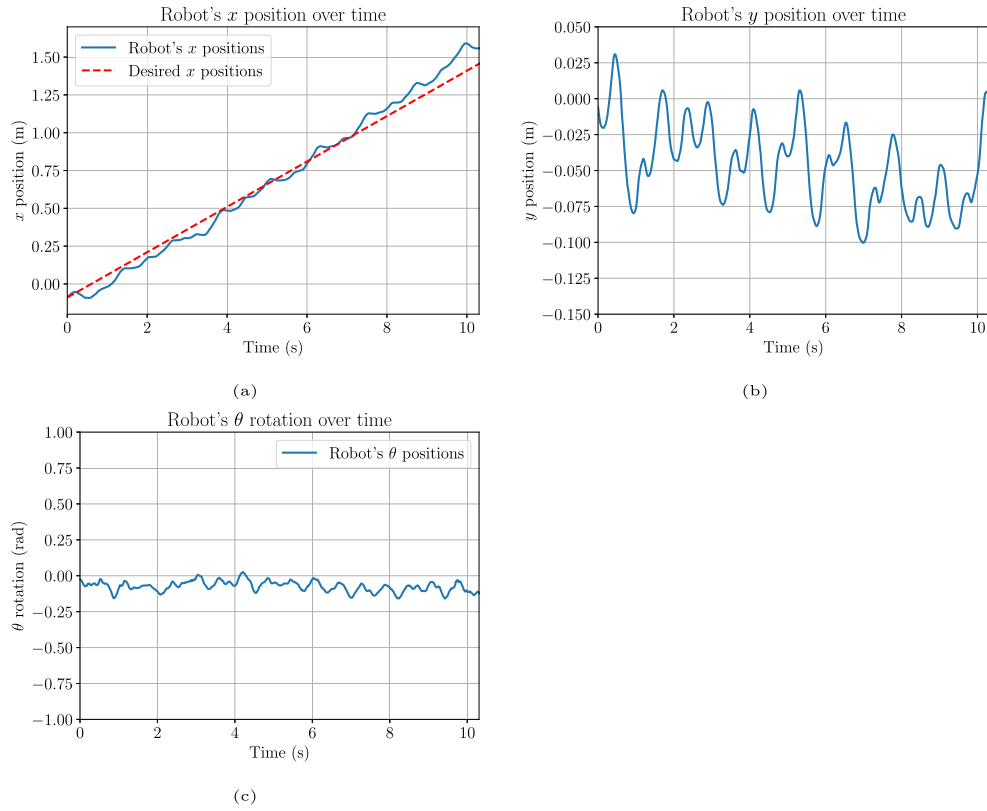


Fig. 15. Real robot walking forward with a reference speed of 0.15 m/s. (a), (b) and (c) show the robot's position over time in the x , y and θ directions, respectively.

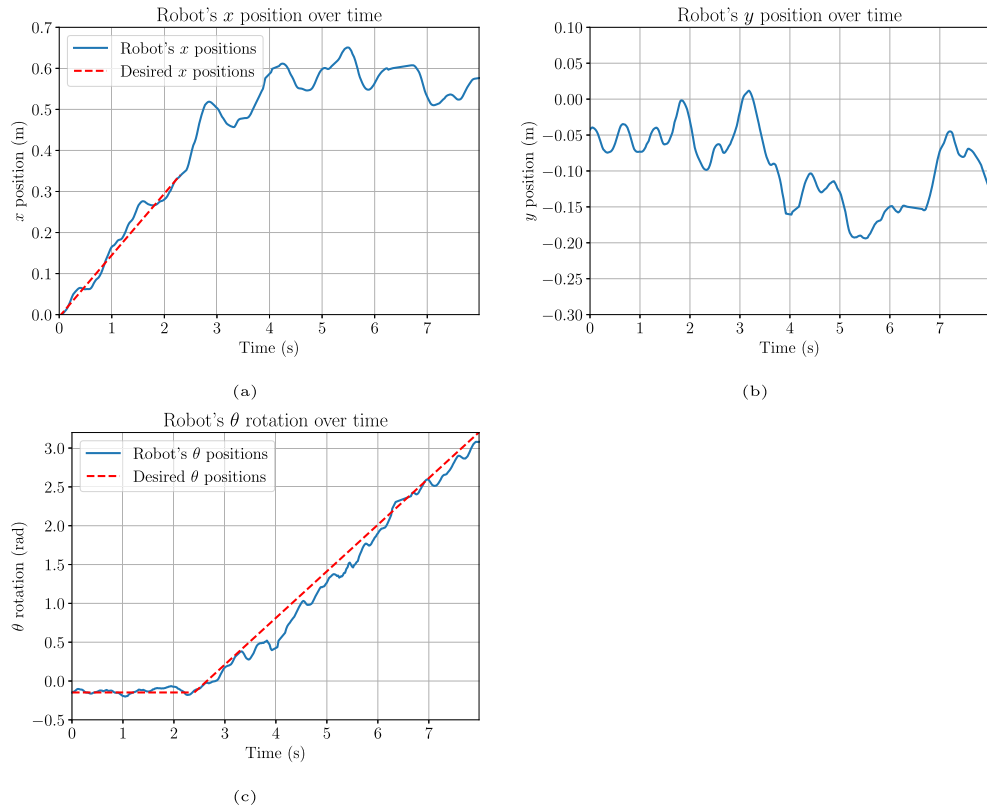


Fig. 16. Real robot walking forward with a speed of 0.2 m/s and then suddenly changing the reference to a 0.5 rad/s rotation. (a), (b) and (c) show the robot's position over time in the x , y and θ directions, respectively.

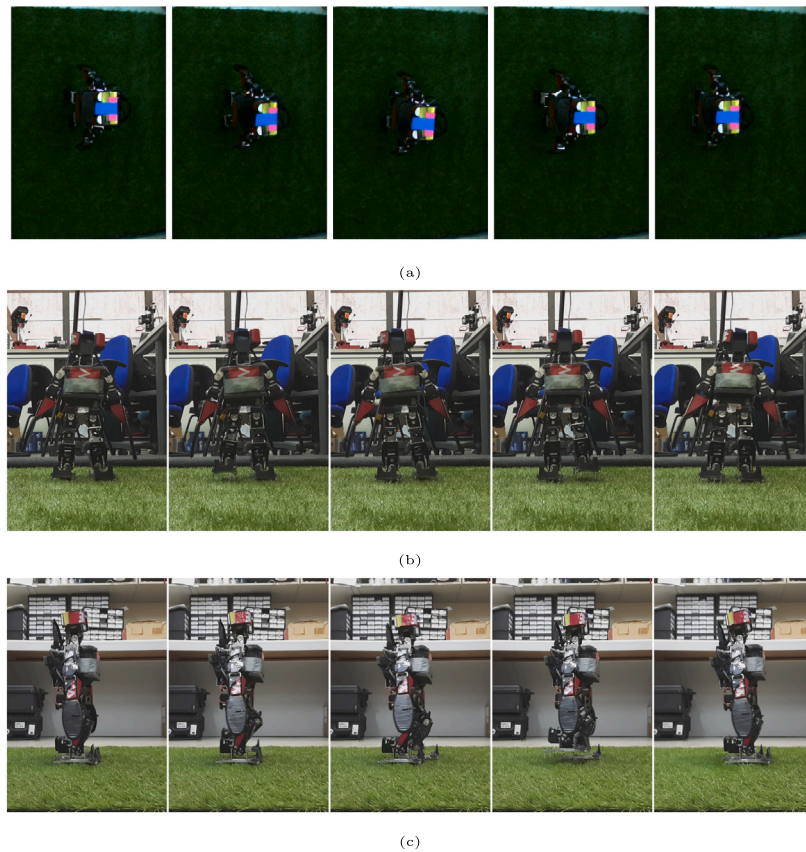


Fig. 17. Snapshots of the Chape robot walking with the proposed IL controller. The robot walks on synthetic grass used in the RoboCup Humanoid KidSize competition: (a) top view, (b) front view, and (c) lateral view.

Potential future research could explore avenues for enhancing the performance of the imitation learning-based algorithm. Techniques like Dataset Aggregation (Dagger) (Ross et al., 2011) may offer improvements in behavior cloning. Additionally, methods in Reinforcement Learning, such as Generative Adversarial Imitation Learning (GAIL) (Ho and Ermon, 2016), could potentially enable the imitation learning algorithm to outperform the expert.

Ethical statements

Non applicable.

CRedit authorship contribution statement

Vítor G.B. de A. Porto: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation. **Dicksiano C. Melo:** Writing – review & editing, Writing – original draft, Software, Formal analysis. **Marcos R.O.A. Maximo:** Writing – review & editing, Writing – original draft, Validation, Supervision, Software, Formal analysis, Data curation, Conceptualization. **Rubens J.M. Afonso:** Writing – review & editing, Supervision, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors thank the sponsors of the ITAndroids' team: Altium, Cenic, Intel, ITAEx, Mathworks, Metinjo, Micropress, Neofield, Polimold, Rapid, SIATT, Solidworks, STMicroelectronics, WildLife, and Virtual Pyxis.

Funding information

Marcos Maximo is partially funded by CNPq – National Research Council of Brazil through the grant 307525/2022-8. Vitor Giudice is partially funded by CNPq – National Research Council of Brazil through the grant 130427/2021-7.

Data availability

The data used in this research can be found along with the source code in <https://gitlab.com/itandroids/open-projects/imitation-learning-walking>.

References

- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G.S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., Levenberg, J., Mané, D., Monga, R., Moore, S., Murray, D., Olah, C., Schuster, M., Shlens, J., Steiner, B., Sutskever, I., Talwar, K., Tucker, P., Vanhoucke, V., Vasudevan, V., Viégas, F., Vinyals, O., Warden, P., Wattenberg, M., Wicke, M., Yu, Y., Zheng, X., 2015. TensorFlow: Large-scale machine learning on heterogeneous systems. Software available from tensorflow.org. <https://www.tensorflow.org/>.

- Aftab, Z., Robert, T., Wieber, P.-B., 2012. Ankle, hip and stepping strategies for humanoid balance recovery with a single model predictive control scheme. In: 2012 12th IEEE-RAS International Conference on Humanoid Robots. Humanoids 2012, pp. 159–164.
- Åkesson, B.M., Toivonen, H.T., 2006. A neural network model predictive controller. *J. Process Control* 16 (9), 937–946. <http://dx.doi.org/10.1016/j.procont.2006.06.001>, <https://www.sciencedirect.com/science/article/pii/S0959152406000618>.
- Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M., 2019. Optuna: A next-generation hyperparameter optimization framework. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. pp. 2623–2631.
- Bain, M., Sammut, C., 1995. A framework for behavioural cloning. In: *Machine Intelligence* 15.
- Bohórquez, N., Wieber, P., 2017. Adaptive step duration in biped walking: A robust approach to nonlinear constraints. In: 2017 IEEE-RAS 17th International Conference on Humanoid Robotics. Humanoids, pp. 724–729. <http://dx.doi.org/10.1109/HUMANOIDS.2017.8246952>.
- Bohórquez, N., Wieber, P., 2018. Adaptive step rotation in biped walking. In: 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems. IROS, pp. 720–725. <http://dx.doi.org/10.1109/IROS.2018.8594431>.
- Carius, J., Farshidian, F., Hutter, M., 2020. MPC-net: A first principles guided policy search. *IEEE Robotics Autom. Lett.* 5 (2), 2897–2904. <http://dx.doi.org/10.1109/lra.2020.2974653>.
- Cavagnari, L., Magni, L., Scattolini, R., 1999. Neural network implementation of nonlinear receding-horizon control. *Neural Comput. Appl.* 8, 86–92. <http://dx.doi.org/10.1007/s005210050010>.
- Chen, S., Saulnier, K., Atanasov, N., Lee, D.D., Kumar, V., Pappas, G.J., Morari, M., 2018. Approximating explicit model predictive control using constrained neural networks. In: 2018 Annual American Control Conference. ACC, pp. 1520–1527. <http://dx.doi.org/10.23919/ACC.2018.8431275>.
- Ding, Y., Khazoom, C., Chignoli, M., Kim, S., 2022. Dynamic walking with footstep adaptation on the mit humanoid via linear model predictive control. *Rn* 1000, 6.
- Dong, F., Li, X., You, K., Song, S., 2022. Standoff tracking using DNN-based MPC with implementation on FPGA. <http://dx.doi.org/10.48550/ARXIV.2212.10945>, <https://arxiv.org/abs/2212.10945>.
- Duan, Y., Andrychowicz, M., Stadie, B.C., Ho, J., Schneider, J., Sutskever, I., Abbeel, P., Zaremba, W., 2017. One-shot imitation learning. *CoRR abs/1703.07326*. <http://arxiv.org/abs/1703.07326>.
- Engelsberger, J., Ott, C., Albu-Schäffer, A., 2015. Three-dimensional bipedal walking control based on divergent component of motion. *IEEE Trans. Robot.* 31 (2), 355–368. <http://dx.doi.org/10.1109/TRO.2015.2405592>.
- Goodfellow, I.J., Bengio, Y., Courville, A., 2016. *Deep Learning*. MIT Press, Cambridge, MA, USA. <http://www.deeplearningbook.org>.
- Griffin, R.J., Wiedebach, G., Bertrand, S., Leonessa, A., Pratt, J., 2017. Walking stabilization using step timing and location adjustment on the humanoid robot, atlas. In: 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems. IROS, pp. 667–673. <http://dx.doi.org/10.1109/IROS.2017.8202223>.
- Gurobi Optimization, Inc., 2015. *Gurobi Optimizer Reference Manual*. <http://www.gurobi.com>.
- Herdt, A., Diedam, H., Wieber, P.-B., Dimitrov, D., Mombaur, K., Diehl, M., 2010a. Online Walking Motion Generation with Automatic Foot Step Placement. *Adv. Robot.* 24 (5–6).
- Herdt, A., Diedam, H., Wieber, P.-B., Dimitrov, D., Mombaur, K., Diehl, M., 2010b. Online walking motion generation with automatic footstep placement. *Adv. Robot.* 24 (5–6), 719–737. <http://dx.doi.org/10.1163/016918610X493552>.
- Hietala, J., Blanco-Mulero, D., Alcan, G., Kyrki, V., 2021. Closing the sim2real gap in dynamic cloth manipulation. 2, arXiv preprint [arXiv:2109.04771](https://arxiv.org/abs/2109.04771).
- Ho, J., Ermon, S., 2016. Generative adversarial imitation learning. In: Lee, D., Sugiyama, M., Luxburg, U., Guyon, I., Garnett, R. (Eds.), *In: Advances in Neural Information Processing Systems*, vol. 29, Curran Associates, Inc., <https://proceedings.neurips.cc/paper/2016/file/cc7e2b878868cbac992d1fb743995d8f-Paper.pdf>.
- Ibanez, A., Bidaud, P., Padois, V., 2014. Emergence of humanoid walking behaviors from mixed-integer model predictive control. In: 2014 IEEE/RSJ International Conference on Intelligent Robots and Systems. pp. 4014–4021. <http://dx.doi.org/10.1109/IROS.2014.6943127>.
- Kajita, S., Hirukawa, H., Harada, K., Yokoi, K., 2014. *Introduction to Humanoid Robotics*. Springer Tracts in Advanced Robotics, New York, United States.
- Kajita, S., Kanehiro, F., Kaneko, K., Fujiwara, K., Harada, K., Yokoi, K., Hirukawa, H., 2003. Biped Walking Pattern Generation by using Preview Control of Zero-Moment Point. In: Proceedings of the 2003 IEEE International Conference on Robotics and Automation. IEEE, Taipei, Taiwan, pp. 14–19.
- Kajita, S., Kanehiro, F., Kaneko, K., Yokoi, K., Hirukawa, H., 2001. The 3D Linear Inverted Pendulum Mode: A simple modeling for a biped walking pattern generation. In: Proceedings of the 2001 IEEE/RSJ International Conference on Intelligent Robots and Systems. Hawaii, USA, <http://dx.doi.org/10.1109/IROS.2001.973365>.
- Karg, B., Alamo, T., Lucia, S., 2021. Probabilistic performance validation of deep learning-based robust NMPC controllers. *Internat. J. Robust Nonlinear Control* 31 (18), 8855–8876. <http://dx.doi.org/10.1002/rnc.5696>.
- Khadiv, M., Herzog, A., Moosavian, S.A.A., Righetti, L., 2016. Step timing adjustment: A step toward generating robust gaits. *CoRR abs/1610.02377*. <http://arxiv.org/abs/1610.02377>.
- Kingma, D., Ba, J., 2014. Adam: A method for stochastic optimization. In: *International Conference on Learning Representations*.
- Kryczka, P., Kormushev, P., Tsagarakis, N.G., Caldwell, D.G., 2015. Online regeneration of bipedal walking gait pattern optimizing footstep placement and timing. In: 2015 IEEE/RSJ International Conference on Intelligent Robots and Systems. IROS, pp. 3352–3357. <http://dx.doi.org/10.1109/IROS.2015.7353844>.
- Kumar, P., Rawlings, J.B., Wright, S.J., 2021. Industrial, large-scale model predictive control with structured neural networks. *Comput. Chem. Eng.* 150, 107291. <http://dx.doi.org/10.1016/j.compchemeng.2021.107291>, <https://www.sciencedirect.com/science/article/pii/S0098135421000697>.
- Lopez, B.T., Slotine, J.-J.E., How, J.P., 2019. Dynamic tube MPC for nonlinear systems. <http://dx.doi.org/10.48550/ARXIV.1907.06553>, <https://arxiv.org/abs/1907.06553>.
- Lovelett, R.J., Dietrich, F., Lee, S., Kevrekidis, I.G., 2018. Some manifold learning considerations towards explicit model predictive control. <http://dx.doi.org/10.48550/ARXIV.1812.01173>, <https://arxiv.org/abs/1812.01173>.
- Maximo, M.R.O.A., 2017. (Ph.D. thesis). Instituto Tecnológico de Aeronáutica, São José dos Campos, SP, Brazil.
- Maximo, M.R.O.A., Afonso, R.J.M., 2020. Mixed-integer quadratic programming for automatic walking footstep placement, duration, and rotation. *Optim. Control Appl. Methods* 41 (6), 1928–1963. <http://dx.doi.org/10.1002/oca.2601>, <https://onlinelibrary.wiley.com/doi/abs/10.1002/oca.2601>.
- Maximo, M.R., Colombini, E.L., Ribeiro, C.H., 2017. Stable and fast model-free walk with arms movement for humanoid robots. *Int. J. Adv. Robot. Syst.* 14 (3), 1729881416675135. <http://dx.doi.org/10.1177/1729881416675135>.
- Maximo, M.R.O.A., Ribeiro, C.H.C., Afonso, R.J.M., 2016. Mixed-integer programming for automatic walking step duration. In: 2016 IEEE/RSJ International Conference on Intelligent Robots and Systems. IROS, IEEE, Daejeon, South Korea, pp. 5399–5404. <http://dx.doi.org/10.1109/IROS.2016.7759794>.
- Maximo, M.R.O.A., Ribeiro, C.H.C., Afonso, R.J.M., 2020. Real-time walking step timing adaptation by restricting duration decision for the first footstep. *Adv. Robot.* 34 (21–22), 1420–1441. <http://dx.doi.org/10.1080/01691864.2020.1821769>.
- Melo, D.C., Máximo, M.R.O.A., da Cunha, A.M., 2020. Push recovery strategies through deep reinforcement learning. In: 2020 Latin American Robotics Symposium (LARS), 2020 Brazilian Symposium on Robotics (SBR) and 2020 Workshop on Robotics in Education. WRE, pp. 1–6. <http://dx.doi.org/10.1109/LARS/SBR/WRE51543.2020.9306967>.
- Melo, D.C., Maximo, M.R.O.A., da Cunha, A.M., 2022. Learning push recovery behaviors for humanoid walking using deep reinforcement learning. *J. Intell. Robot. Syst.* 106 (1), 8. <http://dx.doi.org/10.1007/s10846-022-01656-7>.
- Melo, L.C., Melo, D.C., Maximo, M.R.O.A., 2021. Learning humanoid robot running motions with symmetry incentive through proximal policy optimization. *J. Intell. Robot. Syst.* 102 (3), 54. <http://dx.doi.org/10.1007/s10846-021-01355-9>.
- Minniti, M.V., Farshidian, F., Grandia, R., Hutter, M., 2019. Whole-body MPC for a dynamically stable mobile manipulator. *IEEE Robotics Autom. Lett.* 4 (4), 3687–3694. <http://dx.doi.org/10.1109/lra.2019.2927955>.
- Missura, M., Behnke, S., 2013. Omnidirectional capture steps for bipedal walking. In: 2013 13th IEEE-RAS International Conference on Humanoid Robots. Humanoids, pp. 14–20. <http://dx.doi.org/10.1109/HUMANOIDS.2013.7029949>.
- Nair, V., Hinton, G.E., 2010. Rectified linear units improve restricted boltzmann machines. In: Proceedings of the 27th International Conference on Machine Learning. ICML-10, pp. 807–814.
- Naveau, M., Kudruss, M., Stasse, O., Kirches, C., Mombaur, K., Souères, P., 2017. A reactive walking pattern generator based on nonlinear model predictive control. *IEEE Robot. Autom. Lett.* 2 (1), 10–17. <http://dx.doi.org/10.1109/LRA.2016.2518739>.
- Ng, A., Russell, S., 2000. Algorithms for inverse reinforcement learning. In: *ICML '00 Proceedings of the Seventeenth International Conference on Machine Learning*. ICML-00, pp. 807–814.
- Paulson, J.A., Mesbah, A., 2020. Approximate closed-loop robust model predictive control with guaranteed stability and constraint satisfaction. *IEEE Control Syst. Lett.* 4 (3), 719–724. <http://dx.doi.org/10.1109/LCSYS.2020.2980479>.
- Precup, R.-E., Roman, R.-C., Safaei, A., 2021. *Data-Driven Model-Free Controllers*. CRC Press, Boca Raton.
- Redmon, J., Divvala, S., Girshick, R., Farhadi, A., 2016. You only look once: Unified, real-time object detection. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. pp. 779–788.
- Reske, A., Carius, J., Ma, Y., Farshidian, F., Hutter, M., 2021. Imitation learning from MPC for quadrupedal multi-gait control. In: 2021 IEEE International Conference on Robotics and Automation. ICRA, IEEE, <http://dx.doi.org/10.1109/icra48506.2021.9561444>.
- Ross, S., Gordon, G., Bagnell, D., 2011. A reduction of imitation learning and structured prediction to no-regret online learning. In: Gordon, G., Dunson, D., Dudí, K. M. (Eds.), Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics. In: Proceedings of Machine Learning Research, vol. 15, PMLR, Fort Lauderdale, FL, USA, pp. 627–635, <https://proceedings.mlr.press/v15/ross11a.html>.
- Rosser, K., Kok, J., Chahl, J., Bongard, J., 2020. Sim2real gap is non-monotonic with robot complexity for morphology-in-the-loop flapping wing design. In: 2020 IEEE International Conference on Robotics and Automation. ICRA, IEEE, pp. 7001–7007.
- Salvato, E., Fenu, G., Medvet, E., Pellegrino, F.A., 2021. Crossing the reality gap: A survey on sim-to-real transferability of robot controllers in reinforcement learning. *IEEE Access* 9, 153171–153187. <http://dx.doi.org/10.1109/ACCESS.2021.3126658>.

- Sygulla, F., Rixen, D., 2020. A force-control scheme for biped robots to walk over uneven terrain including partial footholds. *Int. J. Adv. Robot. Syst.* 17, 172988141989747. <http://dx.doi.org/10.1177/1729881419897472>.
- Torabi, F., Warnell, G., Stone, P., 2018. Behavioral cloning from observation. *CoRR* abs/1805.01954. <http://arxiv.org/abs/1805.01954>.
- Vukobratović, M., Borovac, B., 2004. Zero-moment point – Thirty five years of its life. *Int. J. Humanoid Robots* 1 (1), 157–173.
- Wieber, P.-b., 2006. Trajectory free linear model predictive control for stable walking in the presence of strong perturbations. In: 2006 6th IEEE-RAS International Conference on Humanoid Robots. pp. 137–142. <http://dx.doi.org/10.1109/ICHR.2006.321375>.
- Wieber, P.-B., Tedrake, R., Kuindersma, S., 2016. Modeling and control of legged systems. In: Siciliano, B., Khatib, O. (Eds.), *Springer Handbook of Robotics*, second ed. Springer.
- Zaheer, R., Shaziya, H., 2019. A study of the optimization algorithms in deep learning. In: 2019 Third International Conference on Inventive Systems and Control. ICISC, pp. 536–539. <http://dx.doi.org/10.1109/ICISC44355.2019.9036442>.
- Zamfirache, I.A., Precup, R.-E., Roman, R.-C., Petriu, E.M., 2023. Neural network-based control using actor-critic reinforcement learning and grey wolf optimizer with experimental servo system validation. *Expert Syst. Appl.* 225, 120112.
- Zhang, T., Kahn, G., Levine, S., Abbeel, P., 2015. Learning deep control policies for autonomous aerial vehicles with MPC-guided policy search. <http://dx.doi.org/10.48550/ARXIV.1509.06791>, <https://arxiv.org/abs/1509.06791>.
- Zhao, M., Li, J., 2018. Tuning the hyper-parameters of CMA-ES with tree-structured parzen estimators. In: 2018 Tenth International Conference on Advanced Computational Intelligence. ICACI, pp. 613–618. <http://dx.doi.org/10.1109/ICACI.2018.8377530>.